Trivial Relocatability For C++26

Proposal to safely relocate objects in memory

Document #:	P2786R12
Date:	2025-02-10 22:54 CET
Project:	Programming Language C++
Audience:	CWG, LWG

Reply-to:	Mungo Gill <mgill83@bloomberg.net> Joshua Berne <jberne4@bloomberg.net> Corentin Jabot <corentinjabot@gmail.com> Pablo Halpern <phalpern@halpernwightsoftware.com> Lori Hughes <lori@lorihughes.com>

Contents

§ 1 Abstract

Many types in C++ cannot be trivially moved or destroyed but do support trivially moving an object from one location to another by copying its bits — an operation known as *trivial relocation*. Some types even support bitwise swapping, which requires replacing the objects passed to the `swap` function, without violating any object invariants. Optimizing containers to take advantage of this property of a type is already in widespread use throughout the industry but is undefined behavior as far as the language is concerned. This paper provides a mechanism to annotate

types as having the appropriate properties to be eligible for these optimizations, along with library interfaces to make use of them in a well-defined manner.

2 Revision History

R12: February 2025 (mid-Hagenberg)

— We removed the single-object `trivially_relocate_at` function (retaining the bulk `trivially_relocate` function), per LEWG vote in 2025-01-14 telecom.

— LEWG forwarded the library portion of this paper to LWG in their 2025-01-14 telecom.

— We renamed `memberwise_trivially_relocatable` to `trivially_relocatable_if_eligible` and renamed `memberwise_replaceable` to `replaceable_if_eligible` per EWG vote on 2025-02-10 in Hagenberg.

— EWG forwarded the language portion of this paper to CWG in the first session on 2025-02-10 in Hagenberg.

R11: December 2024 (post-Wrocław mailing)

R11 is the third of three revisions in the post-Wrocław mailing.

Although there was some support for each of changes made between R9 and R10 and although those changes were intended to *increase* consensus, some of those changes appeared to *decrease* consensus in the forwarding poll.

— The shortened keywords, despite being the most popular in an LEWG bikeshed poll, resulted in several strongly opposed votes in the forwarding poll because, as Nico Josuttis put it, “This group has to stop giving different semantics the same name.” The lesson we took from that discussion is that keyword names need rationale and should *not* be subject to “bikeshed” discussions. R11 reverts the keywords to those approved by EWG in R9 but leaves keyword naming as an open issue. Alternatives are presented along with a set of principles, including the one articulated by Nico, for choosing reasonable keywords.

— A number of people expressed discomfort with forwarding the low-level interface without knowing the shape of a standard consumer-level interface. R11 restores the consumer-level `relocate` algorithm present in R9 but leaves the choice of consumer-level interface as an open issue, citing an alternative interface described in [P3516R0] (unpublished as of this writing).

— The removal of `swap_value_representations` *did* increase consensus, so it has also been omitted from R11.

In brief, the changes from R10 to R11 are as described below.

— Added a new Proposal Status section that includes a brief summary of the proposal and remaining open issues

— Added a new Open Issues section with recommended resolutions and possible polls

— Restored the keywords to the R9 versions, as approved by EWG

— Added a low-level `trivially_relocate_at` interface to relocate a single object

— Restored the consumer-level `relocate` algorithm that was present in R9

— Added a `is_nothrow_relocatable` trait

R10: November 2024 (Wrocław meeting, LEWG)

R10 is the second of three revisions in the post-Wrocław mailing.

The R10 revision, containing changes made after EWG review in Wrocław, was presented to LEWG on Wednesday afternoon during the Wrocław meeting. With the exception of the keyword changes, the R10 revision contains no language changes and would not, therefore, affect CWG review.

There was some concern that the keywords approved by EWG and the consumer interface described in R9 would create some opposition in LEWG. Since the paper is mostly intended to address a language feature, the library interface was trimmed back to a minimal set of language-support features, and the keywords were shortened, though the new keywords were expected to be discussed and voted on before being finalized.

- Changed keywords from `memberwise_trivially_relocatable` and `memberwise_replceable` to `trivially_relocatable` and `replceable`
- Reduced scope to minimal set of features
  - Removed the consumer-level, non-trivial, `relocate` algorithm
  - Removed the `swap_value_representations` function template
  - Deferred `std::swap` optimizations to `QoI`
- Editorial changes to sync with the technical changes above
  - Removed all content referring to `swap_value_repesentations` and `relocate`
  - Removed most remaining content talking about `swap`
  - Removed Library support for nontrivial relocation
  - Added rationale for how `swap` can be optimized with just this simplified paper
  - Removed FAQ entries that were no longer relevant
  - Added a section on future Consumer APIs

[P2786R10] failed to reach consensus to forward in LEWG in Wrocław.

R9: November 2024 (Wrocław meeting, EWG)

R9 is the first of three revisions in the post-Wrocław mailing.

The R9 revision was discussed in EWG on the Monday of the Wrocław meeting. It differs from the pre-meeting mailing in having more examples and correcting a few, noncontroversial technical issues. [P2786R9] was forwarded by EWG to Core and LWG.

- Added missing precondition to `trivially_relocate` and `relocate`
- Added an extensive set of worked examples to demonstrate primitive functionality
- Added more FAQ entries prompted by reflector discussion
- Fixed core wording
  - Types with deleted destructors can no longer be trivially relocatable or replaceable.
  - To be *eligible for replacement*, a class must have an *eligible* constructor and assignment operator.
- New wording for `swap_value_representations`
- New discussion of contextual keyword renaming

R8: October 2024 (pre-Wrocław mailing)

- Extracted Document Conventions to above the Introduction
- Added Basic Ideas, a higher-level introduction to the new features
- Listed example types for the different type categories defined by this paper
- Corrected implementation of several example functions
- Extended the FAQ
  - What happens if a relocate operation throws?

    - Why is there no `is_trivially_replaceable` trait?
    - Is it UB to mark a nonconforming type as trivially relocatable?
    - Is it UB to mark a nonconforming type as replaceable?
- Explained how to apply the new features to optimize vectors

R7: September 2024 (midterm mailing)

- Significant redrafting since [P2786R6] to better address EWG and LEWG concerns
- Simplified presentation and discussion of trivial relocatability (for detailed history, consult [P2786R6])
- Integrated discussion of `swap`; the present paper supersedes [P3239R0]
- Behavior changes since [P2786R6]
    - User-provided move assignment now prevents a type from being implicitly trivially relocatable
    - The contextual keyword
        - gets a new name, `memberwise_trivially_relocatable`, to better reflect revised semantics
        - is opt-in only
        - deduces relocatability on bases and members
    - No predicate follows the contextual keyword, so no mechanism for opting out is present
    - A new `relocate` function additionally supports nontrivial types and constant evaluation
- Behavior changes since [P3239R0]
    - Clarifies that trivial swappability is based on being replaceable and trivially relocatable
    - Proposes a new contextual keyword, `memberwise_replaceable`
    - Proposes optimizing `std::swap`, using the new properties, and the `swap_value_representations` function

R1–6: January 2023 – April 2024

Early versions of this paper were careful to include comparison and contrast with other papers in this space. That progress is archived by [P2786R6].

The evolution groups requested a clean draft that presents just our proposal and integrates our follow-up papers such that a single coherent design is presented, and revision R7 is the original response to that request.

3 Proposal Status

P2786R12 was forwarded by both EWG and LEWG to their respective wording groups.

The language portion of this proposal was reviewed by EWG and forwarded to CWG and LEWG during the November 2024 Wrocław meeting. Since then, we have asked EWG to reconsider the keyword names (see Keyword selection). EWG did reconsider the keyword names and forwarded the paper with revised names at the 2025-02-10 session of the Hagenberg meeting.

The library portion of this proposal did not reach consensus in Wrocław but was later forwarded by LEWG (with changes) to LWG during the 2025-01-14 telecom.

The short summary of the proposal, presented in the following subsections, has only enough detail to focus discussion on the open issues as of [P2786R11]. This brief recap is intended for those already familiar with the concepts, terminology, and semantics described in the rest of the paper; readers new to this proposal should continue from the Document Conventions section. Since R11, all open issues have been addressed by the appropriate evolution groups. Each issue lists its resolution at the end.

§ 3.1 Class-property specifier keywords

Two new *class-property-specifier*s (formerly *class-virt-specifier*) are added to the Core language as contextual keywords that appear in a class definition after the class name, i.e., in the same syntactic position as `final`:

```
memberwise_trivially_relocatable
```

and

```
memberwise_replaceable
```

R11 Open Issue: These keyword names are fairly long yet not particularly descriptive of their semantics; see the Keyword selection section.

EWG Resolution: Renamed `memberwise_trivially_relocatable` to `trivially_relocatable_if_eligible` and renamed `memberwise_replaceable` to `replaceable_if_eligible`.

§ 3.2 Low-level memory-management Library functions

Two functions are added to the memory management library:

```
template <class T>
  T* trivially_relocate_at(T* location, T* from);
```

to relocate a single item, and

```
template <class T>
  T* trivially_relocate(T* first, T* last, T* result);
```

to relocate a contiguous sequence of items.

R11 Open Issue: Because these are low-level interfaces, only one is technically needed since one can be written in terms of the other, but they have different safety, usability, and efficiency trade-offs. See the Low-level interface section.

LEWG Resolution: The single-object `trivially_relocate_at` function was removed. No change was made to the bulk `trivially_relocate` function.

§ 3.3 Consumer Library interface

Many types that are not trivially relocatable can still be relocated via move and destroy. By providing a higher-level function that performs relocation by whichever mechanism is available and efficient for a given type, relocation is made more convenient and more forethoughtful of future forms of relocation. The proposed `relocate` algorithm is available in a `constexpr` context, unlike the low-level functions described above:

```
template <class T>
  constexpr T* relocate(T* first, T* last, T* result);
```

R11 Open Issue: [P3516R0], which was not complete as of this writing, presents a different consumer interface. See the Consumer interface section.

LEWG Resolution: No changes were made to the proposed feature. [P3516R0] is still forthcoming and will be discussed separately.

§ 3.4 Library Traits for relocatability and replaceability

Three new traits are added to the Standard Library:

```
is_trivially_relocatable<T>
is_replaceable<T>
is_nothrow_relocatable<T>
```

The last trait does not appear in previous revisions of this paper. It was added because some people, including the authors of this paper, believed it necessary to be able to concisely ask the question of whether `relocate` can be invoked safely. The value of this trait is equal to `is_nothrow_move_constructible_v<T> || is_trivially_relocatable_v<T>` today, but its definition can be expanded in the future to include other relocation methods. Code that uses this trait rather than directly querying the other two traits would thus automatically benefit from evolving support for relocation.

R11 Open Issue: If relocation is inherently a nothrow operation, then `nothrow` in the trait name is redundant. See the `is_nothrow_relocatable` trait section.

LEWG Resolution: No changes were made to the proposed feature; this new trait was retained, and its name was unchanged.

4 Open Issues as of R11

2025-02-10 UPDATE: EWG and LEWG have resolved all open issues and forwarded this paper (P2786R12) to CWG and LWG. The now-closed open issues are listed here for historical review and to provide background on their resolution.

4.1 Keyword selection

The keywords `memberwise_trivially_relocatable` and `memberwise_replaceable` are workable but not very descriptive of their actual function, especially in the case of replaceability. In both cases, the keyword doesn't actually establish the property in question but indicates that the property will be deduced for the class if and only if it holds for all subobjects.

In trying to select better keywords, we applied the following four principles.

1. Do not choose a keyword that strongly implies a property unless it actually establishes that property.
2. Verbosity is acceptable within limits.
3. The keywords shouldn't imply something radically different than intended.
4. Relocatability and replaceability are orthogonal concepts; we need not follow the same naming convention for both.

Some people will object to principle 2 because they don't want class declarations to extend past the rightmost column of their display. However, a *class-property-specifier* like `final` is an elegant way to add properties to a class, and we can envision having a rich set of such properties, as other languages do, in the future. Thus, the list of properties will soon not fit on the same line as the class name (as templated base classes often don't already), and we should get used to listing the properties, however long their names, on separate lines:

```
class X
  memberwise_trivially_relocatable
  memberwise_replaceable
  final
  chocolate_flavored
  unemployed
{
  // ...
};
```

The following keywords conform to the principles above.

— `memberwise_trivially_relocatable` and `memberwise_replaceable`: The status-quo keywords, though not precise, do not run afoul of our principles, although `memberwise_replaceable` might be considered

misleading since replacement is not performed member by member.

— `memberwise_relocatable`: Although the keyword cannot establish triviality, it *can* claim that the class can be relocated by relocating each member. If all the class's subobjects are trivially relocatable, it follows that the class as a whole would be. On the downside, this paper has no provision for memberwise nontrivial relocation, though such a capability is envisioned for the future. Replaceability has no such equivalent.

— `trivially_relocatable_if_eligible` and `replaceable_if_eligible`: These keywords say exactly what is meant. Though they are somewhat long, at 33 and 23 characters, respectively, they are each only one character longer than the status-quo keywords.

The following keywords were ruled out based on the principles listed above.

— `trivially_relocatable` and `replaceable`: Both violate principle 1 because neither property is guaranteed by the use of the keyword; i.e., `is_trivially_relocatable_v<T>` can be `false` even if the keyword is used in the declaration of `T`.

— `try_trivial_relocatable` and `try_replaceable`: Both violate principle 3 because "try" implies some relationship to exceptions.

In addition, the prefixes `tentatively_`, `provisionally_`, and `conditionally_` were ruled out because, though adhering to the principles, they are less descriptive than the `_if_eligible` suffix, despite being the same length or longer.

1. Change `memberwise_trivially_relocatable` to `memberwise_relocatable`. The authors of this paper are in favor of this name change.

2. Change `memberwise_replaceable` to `replaceable_if_eligible`. If poll 1 fails to reach consensus, also change `memberwise_trivially_relocatable` to `trivially_relocatable_if_eligible`. Some authors of this paper are in favor of this name change; the rest are neutral.

EWG Poll Results: Consensus to change `memberwise_replaceable` to `replaceable_if_eligible` and to change `memberwise_trivially_relocatable` to `trivially_relocatable_if_eligible`.

The first poll was not taken since discussion indicated little support for choosing a keyword based on unspecified future features. The two polls below were conducted to choose between `trivially_relocatable_if_eligible` and `permit_trivially_relocatability`. The consensus was clearly in favor of the former.

Poll: p2786r11 change `memberwise_trivially_relocatable` to `trivially_relocatable_if_eligible` and `memberwise_replaceable` to replaceable_if_eligible, and proceed as previously approved to CWG for C++26.

SF	F	N	A	SA
19	26	7	4	1

Poll: p2786r11 change `memberwise_trivially_relocatable` to `permit_trivial_relocatability` and `memberwise_replaceable` to `permit_replaceability`, and proceed as previously approved to CWG for C++26.

SF	F	N	A	SA
5	15	20	13	3

§ 4.2 Low-level interface

The original low-level library interface was the three-parameter, contiguous-sequence-based `trivially_relocate` algorithm. This algorithm was a drop-in replacement for `memmove`, which is how many existing libraries achieve trivial relocation, albeit by relying on undefined behavior.

At the November 2024 meeting in Wrocław, LEWG voted to change to a low-level interface that relocates just one object at a time. The rationale was that such an interface was more natural for the lowest-level interface. The authors opposed this change, arguing that the real use case for trivial relocation was bulk relocation and that the single-

object interface would be more dangerous, allowing programmers to easily create undefined behavior by relocating a single object with dynamic type different from its static type or relocating out of a variable with automatic storage duration, with no obvious marker (such as pointer arithmetic or a cast) indicating that the programmer is performing a dangerous operation.

The R11 wording included both interfaces, with the new, single-object, interface renamed to `trivially_relocate_at`, as preferred by its advocates. The original interface is still included because `trivially_relocate` was the primitive interface approved by EWG, and the authors were, therefore, unaware that they would need to defend it in LEWG and thus had not prepared a clear and compelling argument. In addition, new technical information has cast doubt on the wisdom of removing this `memmove`-like interface.

The arguments in favor of the contiguous-sequence-based `trivially_relocate` and against the single-object `trivially_relocate_at` are as follows.

— `trivially_relocate` has field experience. It is a drop-in replacement for `memmove`, which has been used extensively in multiple libraries. `trivially_relocate_at` has no such field experience.

— Experimentation has shown that a loop calling `trivially_relocate_at` will not be optimized into a single `memmove`-like operation. Thus, for the typical use case of bulk relocation, `trivially_relocate_at` is not fit for purpose and cannot be adapted using a simple wrapper. Regardless of one's interface preference, we should not standardize something that cannot be used for its most common function, nor should we — when we have an alternative that works well today — standardize something based on a hope that compilers will improve in one specific area.

— Arrays are naturally a homogeneous collection of complete objects. A single object passed to `trivially_relocate_at` might be a base class subobject. The result of trivially relocating such a subobject is much worse than slicing since the virtual table of the relocated object will incorrectly be that of the derived class.

— If the array-based `trivially_relocate` *is* used to relocate a single object, the needed pointer arithmetic is apparent in code review:

```
Thing x;  // automatic variable; should not be relocated!
trivially_relocate(&x, &x + 1, dest_p);  // dangerous and suspicious
trivially_relocate_at(dest_p, &x);       // less suspicious but equally dangerous
```

— Although situations do exist by which a single object might be relocated, such situations are the exception and yield much less performance benefit than bulk relocation. If a user discovers a need to frequently relocate a single object, they can easily implement a single-object trivial-relocation function as a wrapper around the array-based version.

Ultimately, both interfaces are low-level footguns intended for expert use, and the status quo in this paper is to include both. The authors' position, however, is that `trivially_relocate_at`, compared to the bulk `trivially_relocate` algorithm, is more dangerous, is more difficult to use correctly, and provides very little in terms of optimization.

Recommended Poll (for LEWG): Remove single-object `trivially_relocate_at`. The authors of this paper are in favor of removal.

LEWG Poll Results: Census for removing `trivially_relocate_at` and preserving `trivially_relocate`.

POLL: Remove the single object version as suggested in P2786R11:

```
template <class T>
  T* trivially_relocate_at(T* location, T* source);
```

SF	F	N	A	SA
2	9	9	1	0

POLL: Remove the bulk version as suggested in P2786R11:

```
template <class T>
  T* trivially_relocate(T* first, T* last, T* result);
```

SF	F	N	A	SA
1	1	2	8	9

§ 4.3 Consumer interface

The consumer-level `relocate` function was included in R9 but removed in R10 with the thought that a different consumer-level interface, more similar to the existing `uninitialized_*` algorithms, would be better and that we could increase consensus by reducing the interface to its low-level bare minimum. Unfortunately, several members of LEWG considered this paper incomplete and were uncomfortable approving the low-level interface without knowing what the consumer-level interface would look like, especially since the low-level interface cannot be used in a `constexpr` context.

In the Wrocław meeting, members of LEWG expressed interest in modeling a consumer interface on the `uninitailized_*` functions already in the Standard. As of this writing, no complete proposal has been put forth for the shape of this interface, although Louis Dionne appears to be working on a paper, [P3516R0], that provides the requested `uninitialized_relocate` interface, compatible with but independent of *trivial* relocation. The authors of this paper are uncomfortable with the interface proposed in P3516R0 for the reasons listed below. The original `relocate` interface has been restored in R11 of this paper, not to prejudice the discussion of future alternatives but to provide a simple and proven interface that allows P2786 to move forward.

Our concerns with modeling the interface after `uninitialized_copy` and its ilk are as follows.

— The generality afforded by an iterator (as opposed to pointer) interface is dangerous, overly complicated, and not useful. We see no use case for noncontiguous iterators that wouldn't leave some container in an undestructible state. Further, most input-only iterators would yield UB if used with these interfaces.

— Louis's draft interface is large (ten new functions and overloads), yet most of it is not well motivated. Moreover, even existing `uninitialized_*` interfaces (as recently expanded) are poorly motivated and not useful, e.g., within allocator-aware containers.

Suggested process: Louis's paper is not published. The Standard would not be broken if both interfaces (`relocate` and `uninitialized_relocate`) were adopted (though the redundancy would be strange). Rather than hold up P2786 or vote something incomplete into the working paper, we suggest that we uncouple the progress of P2786 from P3516 by forwarding P2786 with the `relocate` function template in place. If LEWG ultimately decides that P3516 is a better direction, a future revision of P3516 — one that replaces `relocate` with an `uninitialized_relocate` that takes advantage of trivial relocation — can be adopted, and we will work collegially with Louis to make that happen.

No poll was needed, and none was taken by LEWG.

§ 4.4 `is_nothrow_relocatable` trait

The `is_nothrow_relocatable_v<T>` is `true` if `T` can be relocated either via trivial relocation or via a nothrow move constructor and destructor. We assert that relocation is *inherently* a no-fail (and, therefore, nothrow) operation, so the "nothrow" in the trait name is arguably redundant.

The arguments for keeping "nothrow" are as follows.

— The name is consistent with other traits, such as `is_nothrow_move_constructible`.

— If relocation is later discovered to not be inherently nonthrowing, the trait would be well named and would admit the possibility of an `is_relocatable` trait that does not imply a nothrow operation.

The argument for removing "nothrow" is as follows.

— "Nothrow" implies that relocation might be throwing though we believe it must never be such.

Possible Poll (for LEWG): Rename `is_nothrow_relocatable` to `is_relocatable`. The authors of this paper are neutral on this name change.

LEWG Poll Results: Retain this trait with the current name.

POLL: Remove is_nothrow_relocatable trait from P2786R11.

SF	F	N	A	SA
1	1	0	10	3

POLL: Rename is_nothrow_relocatable into is_relocatable in P2786R11.

Outcome: No objection to unanimous dissent. The name stays as is in the paper: `is_nothrow_relocatable`.

5 Document Conventions

5.1 Typography

Throughout this paper, a **bold typeface** will be used for terms defined herein and ***bold italicized typeface*** for terms of art defined herein; the proposed wording, however, will use the conventions of the Standard.

5.2 Definitions

We define a **relocation operation** of a source object as one that ends the lifetime of that object and starts the lifetime of a new object at a new location. Importantly, the destructor is not necessarily run by a **relocation operation**. For types in which move construction and destruction are supported, a relocation can be accomplished by constructing an object in the new location from an xvalue referring to the source object, followed by invoking the destructor of the source object.

We define a **trivial relocation operation** as a **relocation operation** accomplished by performing a bitwise copy of its *object representation* to a new memory location that ends the lifetime of that source object — just as if that (source) object's storage were used by another object (6.7.4 [basic.life][1]p5) — and starts the life of a new object at the new location. Importantly, nothing else is done to the source object; in particular, *its destructor is not run*. This operation will typically be semantically equivalent to a nontrivial **relocation operation** performed via move construction and destruction (though exceptions, while not encouraged, are not expressly forbidden).

We define **replacement** of a target object by a source object as destroying the target object immediately followed by move construction into the location of the target object from the source object. For many types, this operation is semantically equivalent to a move-assignment operation from the source object to the target object.

5.3 Core-language additions

We propose a new Core-language definition for a ***trivially relocatable type***. This new definition is inspired by the recursive nature and handling of special member functions used in the definition of a *trivially copyable* type. A ***trivially relocatable type*** is a scalar type, a ***trivially relocatable class***, an array of such types, or a cv-qualified version of such a type. A class will be implicitly ***trivially relocatable*** if all its bases and members are ***trivially relocatable*** and none of its eligible special member functions are user provided; a contextual keyword will signify that a class might still be ***trivially relocatable*** even if it has user-provided special member functions.

We similarly propose a new Core-language definition for a ***replaceable type*** in which a class will be a ***replaceable class*** if all its bases and members are ***replaceable*** and none of its relevant special member functions are user provided; a contextual keyword will signify that a class might still be a ***replaceable class***, even if it has those user-provided special member functions.

Because **replaceability** is explicitly about the equivalence of assignment with destruction followed by construction, we do not decide that a type is *implicitly* ***replaceable*** when the special member functions selected for those

operations are user provided.

§ 5.4 Library additions

The Standard Library APIs to support **trivial relocation** and **replaceability** comprise

- a type trait to detect **trivial relocatability**
- a function, `trivially_relocate`, that performs relocation on a range of objects by moving their bytes (similarly to `memmove`) while starting the lifetime of the destination objects and ending the lifetime of the source objects
- a user-facing function, `relocate`, that emulates relocation by using the move-and-destroy idiom for types that are not ***trivially relocatable*** and delegates to `trivially_relocate` for types that are ***trivially relocatable***
- a type trait to detect nothrow **relocatability**, i.e., whether a type can be used with `relocate`
- a type trait to detect **replaceability**

To support common use cases, both `trivially_relocate` and `relocate` are specified to support overlapping ranges.

Finally, we propose modifications to Standard Library wording to describe when Standard Library types are allowed and expected to have various properties, including **trivial relocatability** and **replaceability**.

§ 6 Introduction

Containers in C++, in particular those like `std::vector` and `std::deque` that manage objects within a range of continuous storage, live and die by the efficiency with which they can move objects around. One of the most common fundamental steps in many of the operations these types perform is that of relocation — taking an element at one location in memory and creating a new element at a different location in memory with the same value and then destroying that original value.

Many frequently used libraries have long recognized that, for many types, the two nontrivial steps of move construction and destruction often combine into a single operation that can be accomplished by a simple bitwise copy followed by discarding the source object instead of evaluating its destructor. Much of the work a move constructor might do to the source object, such as setting pointers to owned data to `nullptr`, is done only to make sure the destructor that will eventually run knows that no data is present that it is still responsible for freeing. By taking advantage of the knowledge that certain types can be relocated by simply copying bits, complex operations that can involve the invocation of many user-provided special members functions can be replaced by single calls to `memcpy`, realizing huge benefits in performance.

The problem, of course, is that moving objects in this fashion that are not trivially copyable violates the C++ object model and is undefined behavior. In this paper, we propose a mechanism to fix that problem.

- Types with no user-provided special member functions are identified as ***trivially relocatable*** based on whether their bases and nonstatic data members are ***trivially relocatable***.
- Users are able to further identify those types whose user-provided special member functions compound into a trivial operation by marking those types with a new class specifier, `trivially_relocatable_if_eligible`.
- The Standard Library then provides a tool to safely perform **trivial relocation** operations.

A more subtle problem occurs where developers want to apply optimizations based on **trivial relocation**, but their code was previously taking advantage of library APIs to assume that assignment and destroy-then-construct were interchangeable operations. For types that do not support that property, switching to `trivially_relocate`, which emulates move construction, will behave differently than code optimized to use assignment instead. This property — that we name **replaceability** — of a type is not programmatically detectable today. For a frequently mentioned example, the C++ Standard Library specification for `std::vector` insert and erase operations allows implementers to relocate elements using assignment, assuming but not requiring that all stored types are ***replaceable***; for an

implementation using assignment to relocate elements in these operations to start using **trivial relocation** instead — an otherwise valid transformation — would be an observable change of behavior unless the implementation could make a compile-time test to verify that the element type is not only ***trivially relocatable***, but also ***replaceable***.

To incorporate this concept of **replaceability** into the language, we propose further additional changes.

— Types are implicitly recursively ***replaceable*** if all their members and bases are ***replaceable***.

— Through the use of the `replaceable_if_eligible` specifier on a class, users can specify that their user-provided constructors, destructors, and assignment operators still satisfy the appropriate equivalence specified by being ***replaceable*** as long as all members and bases also have this property.

Put together, we hope this proposal provides a complete picture of how to incorporate into the C++ Standard, in a comprehensible and effective manner, bitwise operations that are already performed by many libraries in the industry.

§ 7 Basic Ideas

This paper introduces two new complementary but independent notions into C++.

§ 7.1 Trivial relocatability

Relocation is the act of moving an object and all its nested subobjects from one memory location to another. This result is typically achieved by calling the move constructor to make a new object at the new location followed by calling the destructor of the original object to end its lifetime.

A type is ***trivially relocatable*** if it can be relocated by copying the bytes of its *object representation* from the old location to the new and the lifetime of the original object can be ended *without* running its destructor. However, C++ object lifetimes do not currently permit types (with the exception of those few types that meet the strict requirements of trivial copyability) to be relocated by means of byte copying (**trivial relocation**).

If the object model were to allow it, most C++ types could safely be trivially relocated. The two known exceptions are types that maintain an internal pointer to a data member and types that register their presence in an external registry that must point back to the object.

This paper proposes adding

— a Core-language definition for ***trivially relocatable types***

— a way to explicitly mark types ***trivially relocatable*** when that cannot be deduced by a compiler

— a type trait to report whether a type is ***trivially relocatable***

— a “compiler-magic” function to perform **trivial relocation**, respecting object lifetimes

— a user-facing `relocate` function that can be safely used with types that are not ***trivially relocatable***

— a type trait to report whether a type is nothrow-relocatable

The equivalent of **trivial relocation** has been used for decades with code that has relied on compilers not reacting to the use of undefined behavior when bitwise copying nontrivial types, even though such copies violate C++ object lifetime rules.

§ 7.2 Replaceability

Replaceability is a semantic property of a type, where move assignment is isomorphic to destroy then move-construct. Just like in **trivial relocatability**, a compiler cannot deduce whether a type is ***replaceable*** if the user provides a move-assignment operator, move constructor, or destructor without extra guidance from the user.

In many cases, a library would like to require or assume **replaceability**, such as for moving elements around a `std::vector` when inserting or erasing elements.

This paper proposes adding

— a Core-language definition for ***replaceable types***

— a way to mark types as ***replaceable*** if the compiler cannot deduce that property

— a type trait to report whether a type is ***replaceable***

Note that no part of the language requires types to be ***replaceable***; this feature is purely to allow users to mark their types with a property that many libraries seek to exploit.

§ 7.3 Examples of types having neither, one, or both properties

In the following table, let `list1` be an implementation of `std::list` whose sentinel node is part of the container's footprint and `list2` be an implementation of `std::list` whose a sentinel node is allocated on the heap. Also, let `string1` be an implementation of `std::string` that may contain a pointer to its own small-string buffer and `string2` be an implementation of `std::string` that will never point to itself. These examples are described in more detail in the subsequent subsections.

	Not replaceable	Replaceable

Not trivially relocatable	`pmr::list1<T>`
	`tuple<pmr::string1>`
	`pair<int, const string1>`
		`list1<T>`
		`string1`

Trivially relocatable	`tuple<T&>`
	`pmr::vector<T>`
	`pmr::list2<T>`
		`shared_ptr<T>`
		`future<T>`
		`vector<T>`
		`string2`
		`list2<T>`
		`tuple<vector<T>>`

§ 7.3.1 Examples of trivially relocatable and replaceable types

If we assume the default template arguments, we would expect the following Standard Library types to be both ***trivially relocatable*** and ***replaceable***:

— `std::shared_ptr`

— `std::future`

— `std::vector`

We would expect the following types to be both ***trivially relocatable*** and ***replaceable*** if all their template arguments are both ***trivially relocatable*** and ***replaceable***:

— `std::pair`

— `std::tuple`

§ 7.3.2 Examples of trivially relocatable types that are *not* replaceable

A variety of types, while ***trivially relocatable***, do not maintain the invariants of **replaceability**:

— `std::tuple<T &>`

— `std::pmr` containers (because `pmr` allocators do not propagate on assignment)

— Any class with a trivially-relocatable `const` data member

In many contexts, relocation of such types is desirable, especially in user-defined data structures beyond the reach of the Standard Library.

§ 7.3.3 Examples of replaceable types that are not *trivially* relocatable

Any object directly or indirectly containing pointers to itself would not be trivially relocatable but can certainly be replaceable. Examples include

— Implementations of `std::basic_string` that may contain a pointer to its own small-string buffer (`string1`)

— Implementations of `std::list` that do not allocate a sentinel node (`list1`); the first and last nodes of such an implementation typically point back to the container object itself

— Containers supporting iterators that point back to the container, typically for detecting iterator invalidation during debugging

This category of types would meet preconditions for algorithms in which the semantics of **replaceability** are important, and they might be enforced by the equivalent of Mandates, Constraints, or Preconditions in users' libraries.

7.3.4 Examples of Standard Library types that must defer to the implementation

We would expect the quality of implementation would decide whether the following types are ***trivially relocatable*** *and* ***replaceable*** or are just ***replaceable***:

— `std::basic_string`, depending on whether the short string optimization maintains an internal pointer (`string1` vs. `string2`).

— `std::list`, depending on whether the sentinel node is a nonstatic data member (`list1` vs. `list2`)

7.4 Independent features

From the variety of types and usage examples above, we see that while **trivial relocation** and **replacement** are often used together, each has important use cases and neither can be built on top of the other.

8 Technical Background

Some very specific uses of terminology from the C++ Standard are important to understand when reading this proposal and are quickly summarized here.

For decades, C++ developers have been optimizing low-level data structures, such as their own `vector`-like types, by byte-wise copying objects from one location to another, even though doing so is often UB; see earlier papers[2] for rationale.

Earlier revisions of this paper initially proposed language and library extensions, termed **trivial relocation**, to make writing such code well defined and was forwarded to Core where it received a strong review that challenged our assumptions about copying bytes. From the perspective of the C++ abstract machine, we should not be making assumptions about in-memory representations — that is the compiler's job — and should limit ourselves to copying the *object representation*, leaving the compiler itself to optimize copying and moving the object representations using efficient memory-copying operations.

The Core review in Tokyo 2024 proceeded in parallel to the LEWG review at the same meeting, which subsequently sent the proposal back to EWG, asking for a more complete handling of bitwise operations, notably optimizations for byte-wise swaps. Subsequent feedback, given that `swap` cannot rely on **trivial relocation** lest it corrupt a potentially overlapped member subobject, was that the full details of `swap` are best left to the Library and compiler to work out for themselves and that supplying the two traits for **trivial relocatability** and for **replaceability** is sufficient for them to make progress.

Polls in LEWG in Wrocław 2024 indicated that the fundamental primitive should be an API to relocate a single object at a time, leaving relocating ranges to a higher-level consumer API to be separately proposed in follow-up papers. Further consideration as well as compiler benchmarks indicate that single-object **trivial relocation** has few uses cases, cannot be optimized efficiently by current compilers, and fails to achieve the important goal of being a

drop-in substitute for `memmove`. In deference to the LEWG poll, the single-object primitive was retained, but the contiguous-sequence primitive was also restored.

9 Basic Design Principles

9.1 Create a feature for users, not just the Standard Library

Efficient implementation of many data structures often needs a means to efficiently move and exchange objects that those data structures are managing, especially for data structures that manage their elements in contiguous storage or in some other location that is not a node at the end of a pointer.

We note that such object manipulation is also a sharp tool that is not initially expected to see much use other than optimizing the internal management of data structures. While this paper focuses on supplying the essential building blocks of **trivial relocation**, we defer the design of a more usable consumer API to follow-up papers that can debate their varied merits and trade-offs.

9.2 A formal specification is built around object lifetimes

C++ has a well-specified object model that is important to optimizers, sanitizers, and analysis tools alike. Such tools must reason about object lifetimes and, importantly, minimize the doubt created for developers regarding that reasoning leading to false positives or false negatives when seeking to optimize or alert users.

9.3 Behavior must be reliable

No freedom for quality of implementation (QoI) in semantics is an important quality that builds on the well-specified object model.

9.4 Guard against accidental undefined behavior

The new Library APIs support only types that would produce well-defined behavior. The specification prefers *Mandates* clauses to *Constraints:* clauses since SFINAE behavior carries no expected benefit and is likely to produce error messages with less useful information.

10 Core Proposal

Our core proposal comprises two parts: **trivial relocation** and **replaceability**, including Standard Library primitives that are necessary for well-defined use of each feature. **Trivial relocation** is a technique already widely used in the industry, and **replaceability** is a more novel property that is exploited directly by optimizing library code based on its availability.

10.1 Trivial relocatability

To ensure that libraries taking advantage of the ***trivially relocatable*** semantic do not introduce undefined behavior, the model of lifetimes for objects must be extended to allow for relocation of ***trivially relocatable types***. Since the compiler cannot know if a specific `memcpy` or `memmove` call is intended to duplicate (or to move) an object, we propose introducing a new function template, `std::trivially_relocate`, that is restricted to ***trivially relocatable types***. The purpose of the new function template is to efficiently move the *object representation*, typically with a call to `memmove` or `memcpy` while signifying to the compiler (and other analysis tools) that the lifetime of the new objects has begun — similar to calling `start_lifetime_as` on the destination locations — and that the lifetime of the original objects has ended (without running destructors).

This design deliberately puts all compiler-magic and Core-language interaction dealing with the object lifetimes into a single place, rather than into a number of different `relocate`-related overloads. Note that users are not permitted to copy the bytes to perform a relocation themselves, unlike with trivial copyability, although byte copies would continue to work for trivially copyable types.

§ 10.1.1 New type category: Trivially relocatable

To better integrate language support, we further propose that the language can detect types as ***trivially relocatable*** where all their bases and nonstatic data members are, in turn, ***trivially relocatable***: The constructor selected for construction from a single rvalue of the same type is neither user provided nor deleted, the same applies for the assignment operator for rvalues, and their destructor is neither user provided nor deleted. Conceptually, this definition combines the rules we would follow if there was a new user-definable special member function for relocation *and* when that operation would be trivial.

Note that our notion of relocation relies on being semantically equivalent to move construction of the target followed by destruction of the source. Even though it is not involved in this definition, we still consider assignment operations when deciding if a type is *implicitly* ***trivially relocatable*** for the same reasons that we consider assignment when deciding if a type should have an implicitly declared move constructor; any existing type with a particular set of user-provided special member functions should not begin to have new operations considered valid for it if those operations might subvert expectations due to compiling with a new language Standard.

§ 10.1.2 New keyword and explicit rule

Without an opt-in mechanism, the only types that would be implicitly ***trivially relocatable*** would be those that are already trivially copyable, an important yet relatively small subset of the full universe of types in C++. To enable **trivial relocatability** on the many more interesting types that have nontrivial special member functions, explicitly marking such types must be possible. This marking is needed for only user-defined class types (including unions); hence, we propose adding a new contextual keyword, `trivially_relocatable_if_eligible`, as part of the class definition, similar to how `final` applies to classes:

```
struct X;                                       // Forward declaration does not admit `final`.
struct X final {};                              // Class definition admits `final`.
struct Y trivially_relocatable_if_eligible {}; // New contextual keyword, placed like `final`.
```

We propose one new contextual keyword, `trivially_relocatable_if_eligible`, that can be placed in a class-head (on a class definition) to indicate that a type's special operations do nothing that would violate the implicit rule that would make a type ***trivially relocatable***.

By means of the `trivially_relocatable_if_eligible` specification, a class will be determined to be ***trivially relocatable*** if, according to the implicit rules for a ***trivially relocatable class***, the class would be ***trivially relocatable*** if the presence of user-declared special member functions were ignored.

Users considering whether to apply this keyword to a given type that has user-provided special member functions must simply inspect their move constructor and destructor and decide if, when applied together as part of a relocate operation, they have no net effect. Common examples include many types.

— For a resource-owning type, such as `std::unique_ptr`, the newly constructed object will have the same bits as the source object, the source object will have its pointer member set to `nullptr`, and the source object destructor will do nothing because, by the time it runs, that member will be `nullptr`. Simply copying the bytes and discarding the source object achieves the same semantic effect.

— A reference-counting type, however, might increment a count by one when constructing the target object and then decrement that same count by one when destroying the source object. Combining these operations clarifies that the constructor and destructor negate one another.

§ 10.1.3 New type trait: `is_trivially_relocatable`

To expose the **relocatability** property of a type to library functions seeking to provide appropriate optimizations, we propose a new trait, `std::is_trivially_relocatable<T>`, which enables the detection of **trivial** :

```
template< class T >
struct is_trivially_relocatable;

template< class T >
constexpr bool is_trivially_relocatable_v = is_trivially_relocatable<T>::value;
```

The `std::is_trivially_relocatable<T>` trait has a base characteristic of `std::true_type` if `T` is ***trivially relocatable*** and has `std::false_type` otherwise.

Note that the `std::is_trivially_relocatable` trait reflects the underlying property that a type has, and like all similar traits in the Standard Library, it must not be *user specializable*. Compilers themselves are expected to determine this property internally and should not introduce a library dependency such as by instantiating this type trait.

We expect that the `std::is_trivially_relocatable` trait shall be implemented through a compiler intrinsic, much like `std::is_trivially_copyable`, so the compiler can use that intrinsic when the language semantics require **trivial relocatability**, rather than requiring actual instantiation (and knowledge) of the Standard Library trait. The trait must always agree with the intrinsic since users do *not* have permission to specialize standard type traits (unless explicitly granted permission for a specific trait).

We see no particular need to separately detect whether a type has attempted to make itself ***trivially relocatable*** with the `trivially_relocatable_if_eligible` token or by leaning on the implicit definition.

10.1.4 New primitive function: `trivially_relocate`

As stated in "Library additions," we are proposing a new function template, `trivially_relocate`, that is the primitive entry point into the core magic that tracks and manages object lifetimes in the abstract machine:

```
template <class T>
T* trivially_relocate(T* first, T* last, T* result)
{
  static_assert( is_trivially_relocatable_v<T> && !is_const_v<T> );
  // ... (platform-provided implementation)
}
```

This function template *mandates* that `is_trivially_relocatable_v<T> && !is_const_v<T>` is `true` and has a *precondition* that any objects nested within the source objects are also ***trivially relocatable***.

Its postcondition is that the new objects — and all their nested subobjects — at the destination addresses have the same object representations as the objects — and their corresponding nested subobjects — originally at the source locations; the lifetimes of the source objects and their subobjects have ended without running any destructors or other clean-up code.

On most platforms, this template is functionally equivalent to

```
memmove(result, first, last - first);
```

However, unlike `memmove` on its own, this function template is restricted to ***trivially relocatable types*** rather than to implicit lifetime types.

Note that this function has the nofail guarantee and can never throw an exception, yet it is not marked as `noexcept`, following the principles of the Lakos Rule, which can be summarized as, "If a function has a narrow contract, then, unless that function is likely to be used in conjunction with the `noexcept` operator, the exception specification should be left to the library as QoI."

In addition to performing `memmove`, the function also has the following two important effects that matter to the abstract machine but have no apparent physical effect (i.e., these effects do not change bits in memory), much like `std::launder`.

1. The function ends the lifetime of the source objects. This ending of the objects' lifetimes means that attempting to access those objects or attempting to run their destructor will be *undefined behavior*.

2. The function begins the lifetime of the result objects. If those objects or any of their nested subobjects are unions, they have the same active elements as the corresponding unions in the original objects.

The current library-level mechanism to start the lifetime of an object without invoking a constructor is `std::start_lifetime_as`, a function that works for only implicit lifetime types that must have trivial default constructors. ***Trivially relocatable types***, however, include a much wider range of types, including many that establish and maintain invariants in their special member functions and thus cannot be implicit lifetime types.

A tool for ending lifetimes is similarly unavailable in the Standard Library today. This task can be accomplished by reusing the storage of an object, but that requires modifications of some sort.

The `trivially_relocate` function, therefore, is interacting with the abstract machine in ways that are not currently available. Importantly, for many of the types we are concerned with (e.g., `std::vector`, `std::unique_ptr`, and so on), the component steps of the **relocation operation** are decidedly not trivial, so we are compelled to make this primitive function responsible for the needed compiler magic.

To remove the need for a larger family of functions and avoid overly limiting cases in which **trivial relocation** might be applied, the `trivially_relocate` function is intended to support overlapping source and destination ranges, just like `memmove`. If the ranges are overlapping, the implementation must take care around the management of the lifetime of objects relocated out of or into the overlap.

§ 10.1.5 New library function: `std::relocate`

The function `trivially_relocate` is a sharp tool that requires compiler magic to implement, and the user must write an alternative code path for types that are not ***trivially relocatable***. General relocation that supports both trivial and nontrivial relocation is, however, a subtle and tedious function to implement correctly, and we do not want to force all users to reimplement this function.

Therefore, we propose an additional user-friendly, general-purpose relocation function, `std::relocate`, that will use `trivially_relocate` for ***trivially relocatable types*** and otherwise **relocate** elements by calling the move constructor to move each object, followed by their destructor. This function must correctly order its moves to support overlapping ranges, just like `trivially_relocate`.

In addition, `std::relocate` is `constexpr` to support easy implementation of `constexpr` containers like `std::vector`. Adding such support means that in addition to checking whether a type is ***trivially relocatable*** before calling `trivially_relocate`, we must also have an `if consteval` path that does *not* call `trivially_relocate` during constant evaluation:

```
template <class T>
constexpr
T* relocate(T* first, T* last, T* result)
{
  static_assert(is_trivially_relocatable_v<T>
                || is_nothrow_move_constructible_v<T>);

  // When relocating to the same location or an empty range, do nothing.
  if (first == result) return last;
  if (first == last) return result;

  // Then, if we are not evaluating at compile time and the type supports
  // trivial relocation, delegate to `trivially_relocate`.
  if ! consteval {
    if constexpr (is_trivially_relocatable_v<T>) {
      return trivially_relocate(first, last, result);
    }
  }

  if constexpr (is_move_constructible_v<T>) {
    // For nontrivial relocatable types or any time during constant
    // evaluation, we must detect overlapping ranges and act accordingly,
    // which can be done only if the type is movable.  Note that trivially
    // relocatable types are allowed to have throwing move constructors, and
    // any throwing move that occurs in this branch will cause constant
    // evaluation to fail.
```

```
    if ! consteval {
      // At run time, when there is no overlap, we can, using other Standard
      // Library algorithms, do all moves at once followed by all destructions.
      if (less{}(last,result) || less{}(result + (last-first), first)) {
        T* result = uninitialized_move(first, last, result);
        destroy(first,last);
        return result;
      }
    }

    if (less{}(result,first) || less{}(last,result)) {
      // Any move to a lower address in memory or any nonoverlapping move can be
      // done by iterating forward through the range.
      T* next = first;
      T* dest = result;
      while (next != last) {
        ::new(dest) T(move(*next));
        next->~T();
        ++next; ++dest;
      }
    }
    else {
      // When moving to a higher address that overlaps, we must go backward through
      // the range.
      T* next = last;
      T* dest = result + (last-first);
      while (next != first) {
        --next;  --dest;
        ::new(dest) T(move(*next));
        next->~T();
      }
    }

    return result + (last-first);
  }

  // The only way to reach this point is during constant evaluation where type `T`
  // is trivially relocatable but not move constructible.  Such cases are not supported,
  // so we mark this branch as unreachable.
  unreachable();
}
```

Note that [P3516R0] presents `uninitialized_relocate` and `uninitialize_relocate_backward` templates that provide a different —larger but more general— alternative interface, providing similar functionality to the `relocate` function template.

§ 10.2 Replaceability

In addition to **trivial relocation**, we introduce the orthogonal notion of **replaceability**. An object of type `T` is ***replaceable*** by an object of type `U` if destroying the object of type `T` and reconstructing an object of type `T` in its place from an xvalue of type `U` is equivalent to assigning to the original object of type `T` with an xvalue of type `U`. Note that **replacement** updates an object's value, so `const`-qualified objects are never ***replaceable***.

Replaceability is an important property when we want to transform **relocation** into assignment or vice versa. Containers such as `std::vector` already make a general assumption that all types are ***replaceable***, but other functions, such as `std::swap`, do not make such an assumption, so we provide a mechanism to identify those types that provide guarantees by using this new property.

§ 10.2.1 New type category: Replaceable type

A type `T` is a ***replaceable type*** if every object of type `T` is ***replaceable*** by every other object of type `T`. Note that ***replaceable types*** must be object types; function types, reference types, and `void` are never ***replaceable***.

A cv-unqualified type `T` will implicitly be a ***replaceable type*** if all its bases and nonstatic members are ***replaceable types*** and if it has no user-provided move constructor, move-assignment operator, nor destructor.

10.2.2 New keyword and explicit rule

To enable **replaceability** to be useful for classes with user-provided special member functions, explicitly marking class (including union) types as potentially ***replaceable*** must be possible (just like for ***trivially relocatable types***). To that end, we propose adding a new contextual keyword, `replaceable_if_eligible`, as part of the class definition (mirroring the design of `trivially_relocatable_if_eligible`).

```
struct X;                              // Forward declaration does not admit `final`.
struct X final {};                     // Class definition admits `final`.
struct Y trivially_relocatable_if_eligible {};  // New contextual keyword is placed like
        `final`.
struct Z replaceable_if_eligible {};            // New contextual keyword is placed like
        `final`.
```

A class can be marked with both `trivially_relocatable_if_eligible` and `replaceable_if_eligible`; we expect many uses of `replaceable_if_eligible` to also require `trivially_relocatable_if_eligible`.

10.2.3 New type trait: `is_replaceable`

To expose the **replaceability** property of a type to library functions seeking to provide appropriate optimizations, we propose a new trait, `std::is_replaceable<T>`, that enables the detection of ***replaceable types***:

```
template< class T >
struct is_replaceable;

template< class T >
constexpr bool is_replaceable_v = is_replaceable<T>::value;
```

The `std::is_replaceable<T>` trait has a base characteristic of `std::true_type` if `T` is ***replaceable*** and `std::false_type` otherwise.

Note that the `std::is_replaceable` trait reflects the underlying property that a type has, and like all similar traits in the Standard Library, it must not be *user specializable*. Compilers themselves are expected to determine this property internally and should not introduce a library dependency such as by instantiating this type trait.

10.3 (Not) Optimizing `std::swap`

`std::swap` usage differs significantly from **trivial relocation** in several ways. `std::swap` is an existing well-specified function with a wide contract, and it starts and ends with two valid objects and cannot end the lifetimes of either without vastly changing its current expected behavior; users will have no well-defined way to implement a safe, general purpose, byte-wise `swap`. However, Standard Library implementations are not constrained by simple things like undefined behavior, so vendors will remain free to provide such optimizations as a QoI feature that relies on `is_trivially_relocatable` and `is_replaceable` to spot candidate types; the implementation would still need to rely on compiler intrinsics to avoid the dangers inherent in nontransparent **replacement**, but techniques to evade this problem are known.

Note that revisions R7–9 of this paper included extensive work to guarantee a byte-wise `swap`, but ultimately those extensions were deemed complex, distracting, and nonessential. They might return in a follow-up paper if this paper (P2786) is adopted.

10.3.1 Untyped nested subobjects

A complete object can store a variety of nested subobjects, the obvious case being all its member subobjects, yet nested subobjects can be created in other ways too. For example, if a class has a nonstatic data member that is an array of `std::byte`, a nested subobject with dynamic storage duration can be created in that storage.

When an object is relocated, all its nested subobjects, including those of dynamic storage duration stashed in member arrays, must be relocated too.

11 Simple Worked Examples

To help dispel confusion and misunderstanding, we present a variety of simple classes that illustrate most of the concerns regarding whether a type will be ***trivially relocatable***, ***replaceable***, neither, or both. For reference, we will also note whether such types are trivially copyable as well.

11.1 Exposition-only classes

The following exposition-only classes have their semantics defined by their documentation comments. They are used throughout the rest of this section to illustrate the interaction of the proposed new facilities with both implicit and explicit deduction of the new properties with a relevant variety of data members.

```
struct Empty {};

static_assert(    is_trivially_copyable_v<X>);
static_assert(    is_trivially_relocatable_v<X>);
static_assert(    is_replaceable_v<X>);


struct Non-Trivial {
   // Implementation details are elided.
   // Non-Trivial is neither trivially copyable, trivially relocatable, nor replaceable.
];

static_assert(not is_trivially_copyable_v<X>);
static_assert(not is_trivially_relocatable_v<X>);
static_assert(not is_replaceable_v<X>);


struct Immobile {
   Immobile(Immobile&&) = delete;
   Immobile& operator=(Immobile&&) = delete;

   Immobile() = default;
};

static_assert(not is_trivially_copyable_v<X>);
static_assert(    is_trivially_relocatable_v<X>);
static_assert(not is_replaceable_v<X>);
```

11.2 Rule of zero

```
struct X{};

static_assert(    is_trivially_copyable_v<X>);
static_assert(    is_trivially_relocatable_v<X>);
static_assert(    is_replaceable_v<X>);
```

```
struct X : Empty {};

static_assert(    is_trivially_copyable_v<X>);
static_assert(    is_trivially_relocatable_v<X>);
static_assert(    is_replaceable_v<X>);
```

```
struct X : virtual Empty {};

static_assert(not is_trivially_copyable_v<X>);
static_assert(not is_trivially_relocatable_v<X>);
static_assert(    is_replaceable_v<X>);         // Replaceable types can have virtual bases.
```

```
struct X trivially_relocatable_if_eligible : virtual Empty {};

static_assert(not is_trivially_copyable_v<X>);
static_assert(not is_trivially_relocatable_v<X>);  // Trivially reloctable types never have
                                                   // virtual bases.
static_assert(    is_replaceable_v<X>);
```

```
struct X {
   Non-Trivial data;
};

static_assert(not is_trivially_copyable_v<X>);
static_assert(not is_trivially_relocatable_v<X>);
static_assert(not is_replaceable_v<X>);
```

§ 11.3 Impact of destructors

```
struct X {
   ~X() = default;
};

static_assert(    is_trivially_copyable_v<X>);
static_assert(    is_trivially_relocatable_v<X>);
static_assert(    is_replaceable_v<X>);
```

```
struct X {
   ~X();
};

X::~X() = default;

static_assert(not is_trivially_copyable_v<X>);
static_assert(not is_trivially_relocatable_v<X>);
static_assert(not is_replaceable_v<X>);
```

```
struct X {
   virtual ~X() = default;
};

static_assert(not is_trivially_copyable_v<X>);
static_assert(not is_trivially_relocatable_v<X>);
static_assert(not is_replaceable_v<X>);
```

```
struct X {
   ~X() = delete;
};

static_assert(not is_trivially_copyable_v<X>);
static_assert(not is_trivially_relocatable_v<X>);
static_assert(not is_replaceable_v<X>);
```

§ 11.4 Impact of move constructors

```
struct X {
   X(X&&) = default;
};

static_assert(    is_trivially_copyable_v<X>);
static_assert(    is_trivially_relocatable_v<X>);
static_assert(    is_replaceable_v<X>);
```

```
struct X {
   X(X&&);
};

X::X(X&&) = default;
```

```
static_assert(not is_trivially_copyable_v<X>);
static_assert(not is_trivially_relocatable_v<X>);
static_assert(not is_replaceable_v<X>);
```

Note: This class has an implicitly deleted copy constructor and an implicitly deleted copy-assignment operator.

```
struct X {
   X(X&&) = delete;
};

static_assert(not is_trivially_copyable_v<X>);
static_assert(not is_trivially_relocatable_v<X>);
static_assert(not is_replaceable_v<X>);
```

12 Plans for Library Update

We plan to enable the adoption of these new features in follow-up papers targeting LEWG.

12.1 Immediate updates

In addition to specifying the type traits and Library functions that enable the facilities, we should update the Library frontmatter to indicate whether and how the Library is allowed to use these features to enhance their QoI.

Clearly, under the as-if rule, the Library immediately gets permission to optimize algorithms and functions for ***trivially relocatable*** and ***replaceable*** types where such optimizations are not observable. For example, `std::vector` could optimize many of its operations for such types, given a suitable allocator, such as the default `std::allocator`. No updates to the Library specification are needed for these optimizations, and follow-up papers that suggest changing specifications to allow such optimizations that *would* be observable should be properly directed to LEWG.

The other category of interest is whether Library types themselves can — or should — be ***trivially relocatable*** or ***replaceable***. For example, any implementation of `std::vector` should be able to satisfy the requirements to be both ***trivially relocatable*** and ***replaceable*** for any element type as long as its allocator has those properties; we might want to mandate that `std::vector` is relocatable and/or ***replaceable*** in such cases. Conversely, in the two common implementation strategies for `std::list`, the *sentinel node* is either dynamically allocated or stored directly in the footprint of the `list`. The dynamic node case is always ***trivially relocatable*** and ***replaceable***, but the in-place representation is neither; however, the in-place representation is *nothrow-movable*, whereas the dynamic case must allocate a new node, which can potentially throw. In both cases, **relocation** will never throw, but different trade-offs must be considered when choosing an implementation strategy, and such cases are almost always better left for implementation QoI (especially since ABI concerns might require consideration).

When granting permission for implementations to use keywords that are in addition to those specified by the C++ Standard, we have taken two approaches that we will term the `noexcept` approach and the `constexpr` approach. In the `noexcept` approach, an implementation is granted permission to add `noexcept` specifications to functions as long as those specifications do not invalidate other aspects of the function contract; i.e., an exception specification cannot be added to a virtual function or to a function that is specified to throw exceptions. Conversely, the `constexpr` approach disallows adding `constexpr` to a function that is not declared as `constexpr` in the C++ Standard.

For the purposes of this paper, we believe the minimal necessary specification should use the `noexcept` approach, and we propose the appropriate wording to say so. That choice will allow implementations to experiment with the feature and then provide clear recommendations for specific cases as follow-up LEWG papers.

12.2 ABI concerns

We believe no ABI concerns exist for libraries applying these new features throughout the Standard Library, even as unspecified QoI improvements.

The name-mangling of a type should not depend on whether it is either ***trivially relocatable*** or ***replaceable***. While these properties can be determined through type traits, by definition of being a new feature, no existing code will be SFINAE-enabled on these traits. Updating the internal layout of any Standard Library type to accommodate optimizations using these traits should be unnecessary.

The main concern might be adding constraints to implementation-specific functions used to dispatch to optimized algorithms, such as when growing a vector. In these cases, to avoid introducing new mangled names that would affect link compatibility, `if constexpr` within the dispatching function could be used to enable a fully link-compatible library.

One situation that should be called out is when a library wants to adopt an optimization with an observable behavior change, such as relocating a nonreplaceable type where previously assignment was used. The same concerns would arise as with any other change of unspecified behavior or even a typical bug fix, and library vendors might choose to be conservative and postpone making those QoI changes.

§ 12.3 Specific follow-ups

In a follow-up paper, we intended to propose adding a new specification element, *Class properties*, for any specification related to class properties 11.2 [class.prop]. The Standard Library already makes some effort to specify whether a class must be trivially copyable, standard layout, and so on, and we believe tracking such specification would be more maintainable with a consistent presentation and using a consistent form.

Once we have a *Class properties* element, we can then review all Library classes and decide whether to specify the **trivial-relocatability** behavior for that class, which might be conditional on its template arguments if it is a class template. We might also deliberately defer specifying behavior to allow for implementations making different choices, such as node-based containers allocating their end node vs. storing the pointers in the container's object representation.

Finally, once we have an easy way to document class properties, we might consider making stronger guarantees on existing Library components where such specification would be useful, e.g., clarifying which types are implicit lifetime.

We would propose moving the specification for the following properties in this new element

- *Trivially Copyable*
- *Standard Layout*
- *Implicit Lifetime*
- *Structural*
- *Aggregate*
- *Empty*
- *Bitmask*

along with the two new properties specified in this paper

- ***Trivially Relocatable***
- ***Replaceable***

The following clauses in the Standard Library specification would then include additional notes regarding this new element and updated specification:

- 16.3.2.4 [structure.specifications] — class properties as well as invariants
- 16.3.3.3.5 [customization.point.object] — might be mildly reformulated with the new specification element
- 16.3.3.6 [objects.within.classes] — might be constraining which members may be added

13 Use Cases

13.1 Optimizing `std::vector` at run time

`std::vector` can optimize moving elements into a new buffer by relying strictly on **trivial relocation** when the allocator does not implement `construct` and `destroy`. A library paper targeting the broader issue of optimizing containers for allocators that use the `construct` and `destroy` customization points will follow since that is a concern for more than just **trivial relocation**.

We find that the current specification allows for **trivial relocation** on `insert` and `erase`, although that use might produce a change of semantics that implementations using assignment prefer to avoid. Hence, we will leave the choice to implementers and their interpretation of the specification.

We expect to provide a Library-specific paper to address the semantics of inserting into and erasing from a `std::vector` that is independent of **trivial relocation** concerns and that leans heavily into **replaceability**.

13.2 Optimizing `std::optional` to be trivially relocatable and replaceable

If `std::optional` is implemented with a variant member (anonymous union) and a boolean flag to indicate if the `optional` is engaged, then memberwise determination of both **trivial relocatability** and **replaceability** will produce the correct property. Typical usage might be something like the following example, which clearly shows that any `optional` implementation is going to provide implementations of all the special member functions and thus require use of both contextual keywords.

Original

```
template <class T>
class optional {


   union {
      T d_object;
   };
   bool d_engaged{false};

public:
   using value_type = T;

   ...
};
```

Optimized

```
template <class T>
class optional
      trivially_relocatable_if_eligible
      replaceable_if_eligible {
   union {
      T d_object;
   };
   bool d_engaged{false};

public:
   using value_type = T;

   ...
};
```

Note that to support the `constexpr` operations required by the Standard, a union-based implementation is the only known way to conform. However, if we were not concerned about `constexpr` evaluations, then we might choose to store our active element in an array of bytes. Unfortunately, adding the `trivially_relocatable_if_eligible` or `replaceable_if_eligible` properties to the class definition will give our class that same property — *even when the array member is used as storage for a type **without** those properties* — since an array of `std::byte` is both ***trivially relocatable*** and ***replaceable***.

This problem can be resolved in several ways, but the key is to include a data member that is *conditionally **trivially relocatable*** or ***replaceable***. This resolution is most easily achieved by adding, to the class, an empty data member that ideally can preserve the object layout and ABI.

```
template <bool = true>
struct OptionallyRelocatable {};

template <>
struct OptionallyRelocatable<false> {
    ~OptionallyRelocatable(){}
```

```
};

static_assert( std::is_trivially_relocatable_v<OptionallyRelocatable<>>);
static_assert(!std::is_trivially_relocatable_v<OptionallyRelocatable<false>>);

static_assert( std::is_replaceable_v<OptionallyRelocatable<>>);
static_assert(!std::is_replaceable_v<OptionallyRelocatable<false>>);
```

Original

```
template <class T>
class optional {


   alignas (T)
   std::byte d_object[sizeof (T)];

   bool      d_engaged{false};




public:
   using value_type = T;

   ...
};
```

Optimized

```
template <class T>
class optional
      trivially_relocatable_if_eligible
      replaceable_if_eligible {
   alignas (T)
   std::byte d_object[sizeof (T)];
   union {
      bool   d_engaged{false};
      OptionallyRelocatable<
         std::is_trivially_relocatable_v<T>
      &&
         std::is_replaceable_v<T>> _;
   };

public:
   using value_type = T;

   ...
};
```

Note that in the above implementation, even though we have made a `union` to contain our empty conditionally relocatable object, the `d_engaged` member will always be active. A similar conditional ***replaceable*** object would have the same implementation and be simple to add as well.

§ 14 Implementation Experience

An implementation of the R9 version of this proposal is available as a fork of Clang and can also be accessed on Compiler Explorer. We have not yet updated that implementation to relocate a single object at a time.

In addition to the handling of the new keywords and class properties, the implementation relies on built-in type traits for `is_trivially_relocatable` and `is_replaceable`, which are not different than other type traits of the same nature.

Our Clang implementation of `trivial_relocate` is implemented in terms of `memcpy`. We did not add the necessary machinery to end and start lifetimes since that task is unsupported by the Clang front end and the LLVM optimizer (a known deficiency of LLVM rather than with our implementation). In general, starting and ending lifetimes requires an implementation to add some optimization fences so that optimizers that perform type-based alias analysis are not overly eager and inappropriately prune all code that depends on the new object lifetimes. Either way, adding such fences to an implementation that supports `start_lifetime_as` would present no notable challenges. We have not explored whether sanitizers would need to be made aware of these function semantics.

Note that Clang already supports the notion of ***trivially relocatable types*** in production, although with no opt-in mechanism. This property is used in the implementation of `std::vector` in libc++ (once again demonstrating an industry need for this feature, as well as deployment experience with very similar ideas).

Clang also offers a `[[clang::trivial_abi]]` type attribute that allows a type to be passed in registers when its destructor/constructor pair can be replaced by a `memcpy`. Types with that attribute can be passed in a register, which affects calling convention, and therefore ABI.

§ 15 FAQ

15.1 Is `void` trivially relocatable?

No, nor is it trivially copyable.

15.2 Are reference types trivially relocatable?

No, nor are they trivially copyable.

Taking the address of a reference to pass it to `trivially_relocate` is not possible. How the compiler implements references is entirely unspecified and might not need physical storage if the reference never leaves a local scope. Asking about copying or relocating a naked reference, rather than the entity it refers to, is not meaningful, so these trivial properties are `false`.

15.3 Why can a class with a reference member be trivially relocatable?

A class with a reference member can be ***trivially relocatable*** for the same reason such a class can be trivially copyable. Strictly speaking, reference members are not nonstatic data members, and we cannot create a pointer-to-data-member to one; they deliberately escape the relevant wording by not appearing in the list of disallowed entities, despite not being trivially copyable or ***trivially relocatable*** as a distinct type in their own right. This wording is subtle and can entrap the unwary but has been standard practice for many years.

15.4 Are *cv*-qualified types, notably `const` types, trivially relocatable?

Yes, if the unqualified type is ***trivially relocatable***.

15.5 Can `const`-qualified types be passed to `trivially_relocate`?

No. While `const`-qualified types are ***trivially relocatable*** and thus do not inhibit the **trivial relocatability** of a wrapping type, they are typically not safe to **relocate** due to leaving behind a dead object that cannot be replaced using well-defined behavior. Hence, the `trivially_relocate` function is constrained to exclude `const`-qualified types. This exclusion can be skirted using `const_cast` if doing so would not introduce undefined behavior.

15.6 Can types that are not implicit-lifetime types be trivially relocatable?

Yes, and our experience tells us to expect the majority of types, even those that own resources and have nontrivial move constructors and destructors, to still be ***trivially relocatable***.

15.7 Why are virtual base classes not trivially relocatable?

Because they are not trivially copyable and because the implementation of virtual base classes on some platforms involves an internal pointer, virtual base classes are not ***trivially relocatable***.

We believe that implementing virtual bases such that trivial copyability and relocatability would not be a concern is possible since all the needed data for indirection could be stored as offsets instead of direct pointers. However, whether all implementations could use such a layout or are able to switch to such a layout is unclear. Forcing this support might also require an ABI break.

In our opinion, this low-level behavior should be kept consistent across platforms, rather than left as an unspecified QoI concern, since our current experience has not yet turned up a usage of virtual base classes that would also benefit from this feature.

We would be happy to remove this restriction, but consistency must be maintained with the corresponding restriction on trivially copyable. If no current ABIs are affected, we might consider normatively allowing — or even encouraging — such an implementation (for both trivialities) as conditionally supported behavior on platforms that would not incur an ABI break.

Note that no issues occur with virtual functions since virtual function-table implementations do not take a pointer back into the class, so the vtable pointer can be safely relocated.

§ 15.8 What happens if a relocate operation throws?

Relocation operations must be no-fail, so they do not permit exceptions; if a relocate operation were allowed to fail, whether the failed state had 0, 1, 2, or more valid objects would be unknowable, essentially leaving the program in an undefined state that cannot be cleaned up correctly, which is a significant problem with objects holding resources like a locked mutex.

Our proposal makes clear that `std::trivial_relocate` cannot fail, and the nontrivial implementation of `relocate` mandates that the object type is *nothrow* move constructible. Hence, neither of our operations can fail by throwing an exception.

§ 15.9 Why do deleted special members inhibit implicit trivial relocatability?

Initially, we considered allowing **trivial relocation** of types with these special members functions deleted, based on a notion that we have been familiar with since C++17 when *mandatory copy elision* started propagating noncopyable and nonmovable return values. However, relocation is not the same as copy elision, so objections arose to the idea that, when a user deliberately removes an operation, we should not *silently* re-enable it via a backdoor method. Note that this inhibition changes only the default, preventing accidental relocation of noncopyable or nonmovable types for which **relocatability** was neither considered nor intended; if **trivial relocatability** is desired, such classes can be made explicitly ***trivially relocatable*** by means of the `trivially_relocatable_if_eligible` keyword.

This design also follows that of the Core language for trivial copyability, which was changed by [CWG1734] to exclude types that deleted all copying operations and which landed in C++17.

§ 15.10 Can the compiler transform argument passing with trivial relocation?

As currently specified, we do not yet enable such support. We believe that this could be accomplished with the appropriate allowances (which already exist for trivially copyable types), but significant work in platform ABIs would be needed to make this happen, similar to what is needed to support Clang's `[[trivial_abi]]` attribute.

To enable bitwise parameter passing, such as through registers, for ***trivially relocatable types***, we would need to enable the compiler to freely create extra instances of our objects when passing arguments and return results from functions, which would then enable a compiler to pass the data itself via a register. Importantly and unlike for trivially copyable types (which have trivial destructors), major changes would be needed to ensure that the receiver of the final object is aware that it is now responsible for destruction of that object since currently the creator of parameters is responsible for their destruction on many ABIs.

A separate proposal for argument passing by relocation was offered in [P2839R0] but was not reviewed favorably on its initial presentation to EWG.

§ 15.11 Can the Standard Library containers use this new feature internally?

Yes, where the current specification is permitted to use move construction to **relocate** an object (e.g., when growing or when moving objects within a `vector`), this feature can be used instead for ***trivially relocatable types***.

A common misconception implies that `vector` is required to use assignment when inserting into or erasing from a `vector` (other than at the back). This requirement is not, however, explicitly specified in the Standard. The misunderstanding stems from a number of places, which are addressed individually in the subsections below.

However, even if an implementation is *allowed* to switch from assignment to relocation for arbitrary ***trivially relocatable*** types, it would likely choose to do so for only such types that are also ***replaceable*** to avoid silently changing behavior for customers relying on such types.

§ 15.11.1 Complexity constraints

The first source of this misunderstanding is that people incorrectly consider the requirement to be implied from (23.3.11.5 [vector.modifiers]p5), which states for `vector::erase`:

> *Complexity:* The destructor of `T` is called the number of times equal to the number of the elements erased, but the assignment operator of `T` is called the number of times equal to the number of elements in the `vector` after the erased elements.

This complexity existed in C++98, and the only revision has been a change in C++11 where the text "assignment operator" was updated to "move assignment operator." Note that `vector::insert` has no such complexity requirement; it is specified only for the `vector::erase` operation.

The misconception also comes from the following sentence in (16.3.2.4 [structure.specifications]p7):

> Complexity requirements specified in the library clauses are upper bounds, and implementations that provide better complexity guarantees meet the requirements.

This statement is *not*, therefore, a mandate from the Standard that calls to `vector::erase` shall use the assignment operator as long as the implementation performs as well as or better than the specified complexity. Given that the `trivially_relocate` function as specified in this paper is guaranteed to perform a copy of bytes of the object representation, it must outperform the complexity requirement, and the Standard, therefore, permits implementations to use the `trivially_relocate` function for `vector::erase` operations.

§ 15.11.2 Precondition specifications

The second source of this misunderstanding stems from phrases such as the following in(23.2.4 [sequence.reqmts]p29):

> `a.insert(p, rv)`
>
> Preconditions: `T` is *Cpp17MoveInsertable* into `X`. For `vector` and `deque`, `T` is also *Cpp17MoveAssignable*.
>
> Effects: Inserts a copy of `rv` before `p`.

Although this specification requires that statements of the form `t = rv` be well-formed, it does *not* impose any limitations on implementations to use assignment when moving objects around internally.

Although the requirement that a type be *Cpp17CopyAssignable* or *Cpp17MoveAssignable* does impose semantic requirements on the assignment operator(s), the requirements are vague and specified in terms of a notion of "value" that is not defined in the Standard; see (16.4.4.2 [utility.arg.requirements]tab:cpp17.moveassignable). This requirement was added in C++11 and has not been revisited since then.

The above explanation refers to `vector::insert(p, rv)`, but the same argument applies to similar preconditions on other member functions. Observe that the postconditions are identical for all sequence containers, including those, such as `list`, that do not require *Cpp17MoveAssignable* as a precondition.

§ 15.11.3 Conclusion

In other words, although most implementations of `vector::erase` and `vector::insert` currently use assignment, which is generally assumed the most efficient approach currently available, implementations are under no obligation whatsoever to do so. The various member functions of `vector` guarantee only that values will be moved around but grant implementations complete freedom as to how that action should be performed, whether by means of (move)

assignment, (move) construction, or any other mechanism. Implementations will, therefore, be permitted to perform this move by means of `trivially_relocate` for types that are ***trivially relocatable***.

In fact, some implementations avoid using assignment for some operations (for reasons of efficiency); see the linked examples for GCC and LLVM.

Note that all the comments above apply equally to `deque` as well as to `vector`.

Note also that this lack of a clear requirement exposes an existing ambiguity for `vector::insert` and `vector::erase` operations where, for the contained type, move-assign plus destroy is not equivalent to destroy plus move-construct. That ambiguity is an issue that exists at the moment, and while we might address it with a future, orthogonal proposal, a solution is not required for **trivial relocation** as specified by this paper. Similarly, we might choose to clarify the complexity and requirements clauses above at some point in the future, but that clarification is not required by this proposal and has been left for another time.

§ 15.12 Do implementations need to mark classes `trivially_relocatable_if_eligible` to benefit?

No, although some classes will need to be annotated to qualify as ***trivially relocatable***. For example, the most common implementations of `std::array`, `std::pair`, and `std::tuple` will be implicitly ***trivially relocatable*** if all their members are ***trivially relocatable***. `std::vector` can safely be marked as ***trivially relocatable*** if its allocator and pointer types are ***trivially relocatable***. `std::list` might be marked as ***trivially relocatable*** if it allocates its tail node but not if the tail node is embedded in the object representation itself.

Once we establish a policy of how much we want to guarantee and how much we want to leave open to implementer choice, a follow-up paper will address desired guarantees for **trivial relocatability** in the Standard Library.

§ 15.13 Can I mark as trivially relocatable a type that is not replaceable?

Yes! For example, this would be appropriate for types having data members that are references or using `std::pmr::polymorphic_allocator` or any other type that does not propagate on `swap`.

§ 15.14 Can I mark a type as replaceable but not trivially relocatable?

Yes! This proposal does not offer any immediate advantages for doing so, but we expect to build on **replacement** to optimize other features, such as assignment, in the future.

§ 15.15 What happened to the predicates for the contextual keywords?

An earlier version of this proposal included the option to add a predicate following each of the new contextual keywords to activate or inhibit their behavior. This feature was dropped for introducing too much complexity, including a new vexing parse to resolve, and having vague semantics when the predicate is `false` but the implicit specification would have been `true`. Given the rarity of such cases and the relative simplicity of the workaround above, we chose to keep the core proposal as simple as possible, following EWG guidance.

§ 15.16 Why is copy-replacement unsupported?

In practice, only **replaceability** of objects of type `T` from xvalues of type `T` seems relevant to the operations we are likely to optimize. We have, therefore, simplified the design to focus solely on such **replacement** (which could be termed move-replacement were we being pedantic) and not overcomplicate the language or users' lives by adding even more properties to consider.

§ 15.17 Why is marking a class that can *never* be trivially relocatable not ill-formed?

A class with a virtual base class can never be ***trivially relocatable***, so why is adding the `trivially_relocatable_if_eligible` identifier to that class not ill-formed?

This case is still well-formed, but the class will indeed never be ***trivially relocatable***, and the type trait will deterministically always return `false`. However, this type might also be used as a base class or data member when instantiating a class template, and we do not want to add complexity by considering such special-case instantiations as well-formed when the original case need not be marked as ill-formed.

However, the deterministic case of a direct virtual base class would make for a useful compiler warning. The more general case of a data member or nonvirtual base class not being relocatable (or ***replaceable***) is deliberately not an error since we want to support different implementations of the same type that have different properties; e.g., different implementations of `std::list` choose different trade-offs on how to store the sentinel node marking the end of the list, yet some of those choices are ***trivially relocatable*** and some are not. We want to avoid the inconsistency of deterministically flagging an error when compiling a class with a `std::list` data member in some Standard Library implementations and not in others.

§ 15.18 Why is there no `is_trivially_replaceable` trait?

A common use case is to require types that satisfy both `is_trivially_relocatable<T>` and `is_replaceable<T>`. We could consider whether this use case occurs frequently enough that adding another trait that is the logical conjunction of the two would be valuable.

We opted to omit this trait from our proposal since such a trait is not primitive to the Core-language design of this paper and could easily be added as an amendment in an LEWG follow-up paper well within the timeframe of C++26 if desired.

The lack of a core type category named *trivially replaceable* is another reason to defer to a follow-up paper, and we would be consuming that potential for future vocabulary for a pure Library extension. Making that choice before advancing this paper is unnecessary.

Finally, we must recognize that a type that is both ***trivially relocatable*** and ***replaceable*** does not have a *trivial* replacement operation. The functionality that such a type enables is to turn a rotate or shift operation into a bitwise one without a change in semantics compared to using assignment for such an operation, but no single **replacement** operation is a bitwise one since that would fail to free resources owned by the original object in the target location.

§ 15.19 Is it UB to mark a nonconforming type as trivially relocatable?

First, the compiler has no way to validate that our class's constructors and destructor do not maintain an invariant that is not relocatable, so the compiler will trust us and enable the type trait. This in itself is not UB, but UB will likely follow when some library code makes a transformation that causes our invariant, such as an internal pointer, to no longer hold. Such UB will occur in the subsequent library call, not in the class definition.

§ 15.20 Is it UB to mark a nonconforming type as replaceable?

Just as erroneously marking a type as ***trivially relocatable*** can lead to undefined behavior in library calls, so can marking a type as ***replaceable***. However, where **replaceability** is used as a constraint without **trivial relocation**, there remain reasonable implementations that do not incur UB. For example, if operations are logged, then the act of writing to a log is typically an observable side effect. Library code that transforms between assignment and destroy-then-construct will have an observable change of behavior, such as the suggested logging, but such changes do not in themselves constitute undefined behavior. The creator of the affected class must decide whether a change of such logging behavior would be problematic and then choose whether to mark their type as ***replaceable***.

§ 15.21 Why does replaceability not require trivial relocatability?

While all specified uses of `is_replaceable` in this proposal require that the type be both ***replaceable*** and ***trivially relocatable***, the principle underpinning **replaceability** — i.e., a consistent definition for constructors, destructor, and assignment operators — is highly relevant in a variety of places in the Standard Library. We anticipate this distinct

trait being useful to Library implementations today, and we expect to see wider adoption in the Standard Library specification once the trait becomes available. For example, `std::vector` expects — but does not require — that its members be ***replaceable*** to efficiently switch to assignment rather than destroy/construct when replacing its elements during an insert or erase operation. Motivating examples for why we might want to address this design are found in [P2959R0], although the specification of **replaceability** in this paper is now the preferred direction rather than the suggestions proposed in that paper.

§ 15.22 Why does trivial relocation support `const` data members, but replacement does not?

Relocation creates new objects and can safely copy `const` members. **Replacement** overwrites the data in the replaced object, which cannot — and should not — replace `const` data.

§ 15.23 Why does [library.class.props] explicitly call out permission to use the contextual keywords?

For the same reason we explicitly grant permission to add `noexcept` to function declarations, even before the exception specification entered the type system, and for the same reasons that implementations cannot experiment with marking functions as `constexpr` due to the observable nature with a (deliberate) lack of explicit permission.

§ 15.24 Why are classes with virtual base classes “replaceable”? What does that even mean?

Classes with virtual bases might be ***replaceable*** but will never be ***trivially relocatable***; just as with trivial copyability, we cannot, at this point, restrict implementations from using implementation strategies for virtual bases that require having self-referential pointers (instead of offsets) that would be invalid if simply copied to a new object.

On the other hand, **replaceability** is a relationship between a type’s constructor, destructor, and assignment operator, all of which are applicable to reason about even for a type with a virtual base class.

In practice, we expect **replaceability** to come into play most often once types like `std::vector` start to prefer relocation (even if not trivial) and use **replacement** (and assignment operators) only for types that declare, by being ***replaceable***, that such a strategy is viable. Not allowing such freedom for a vector of objects with virtual base classes would be counterproductive.

§ 16 Illustrative Examples

§ 16.1 Unconstrained vector

Let us consider the case of a user-written container, similar to `std::vector`. Since `std::relocate` is a nofail function that exploits **trivial relocation** where it is available, we have to consider only two kinds of elements:

— Types that are either ***trivially relocatable*** or no-throw move constructible

— All other types

An alternative summary of these two kinds are

— types that can be safely relocated

— types that cannot be safely relocated

The first case to optimize is relocating elements when the current capacity is exceeded by an insert operation. In this case, we clearly can simply relocate for those element types that are safely relocatable and must manually move-construct the second category, accounting for a possible thrown exception on move.

The next operation to consider is erasing an element. In this scenario, we will destroy the requested element(s) and then, for types that can be safely relocated, `relocate` the tail of the vector to the lower address since `relocate` is nofail and supports overlapping ranges. For the second category of types, we must perform the manual relocation and clear the remains of the tail if an exception is thrown.

The final operation to consider is an insertion in the middle of this vector. Here, the first thing we do, assuming capacity is not exceeded, is relocate all elements from the insertion point up by a distance to allow all the new elements to be inserted. Then we construct all the new elements, which is a potentially throwing operation. If an exception *is* thrown, we have several options for our custom vector. For the strong exception safety guarantee, we can destroy the newly inserted items and then safely relocate the original elements back in place since `relocate` is a nofail operation. Alternatively, we provide the basic guarantee by destroying the old tail — and potentially the newly inserted items — before adjusting the vector's size, or maybe we could even clear the whole vector.

Note that all these operations use only **trivial relocation** and never call for **replaceability**.

§ 16.2 Standard vector

When we add the constraints that the Standard imposes on `std::vector`, we find that **replaceability** becomes a useful property. For both insertion and erasure, the Standard likes to assume that elements are ***replaceable***, i.e., assignment is interchangeable with destroy-then-move-construct. Within that guarantee, the Standard Library vector can use relocation per our custom vector example, but for types that are relocatable but not ***replaceable***, matters become more complicated. That topic will be the subject of a separate paper specific to vector, which is necessary regardless of whether we support relocation in C++26. Having the ability to detect ***replaceable*** types would be extremely helpful for that follow-up paper.

§ 16.3 Conforming implementation of a trivially relocatable `std::optional`

The following implementation of `optional` satisfies the C++ Standard specification for the members that it implements and provides a minimal test driver. This implementation uses the new feature macro to ensure that the code compiles with both C++23 and C++26 and is ***trivially relocatable*** if and only if its element type is ***trivially relocatable***.

To implement the `constexpr` members, the implementation is required to use a union to represent its internal state when engaged[3]:

```
#include <cassert>
#include <iostream>
#include <memory>
#include <new>
#include <type_traits>
#include <utility>

template <class T>
class optional
      trivially_relocatable_if_eligible
      replaceable_if_eligible
{
   union {
      T d_object;
   };
   bool      d_engaged{false};

   constexpr T const * address() const noexcept
   { return ::std::addressof(d_object); };

   constexpr T       * address()       noexcept
   { return ::std::addressof(d_object); };

   template<class... Args>
   constexpr void do_emplace(Args&&... args) {
```

```
      ::new(address()) T(std::forward<Args>(args)...);
      d_engaged = true;
   }

public:
   using value_type = T;

   constexpr optional() noexcept {}

   constexpr optional(optional const & other)
   : d_engaged{other.d_engaged} {
      if (d_engaged) {
        ::new(address()) T( other.value() );
      }
   }

   constexpr optional(optional&& other)
       noexcept(std::is_nothrow_move_constructible_v<T>)
   : d_engaged{other.d_engaged}
   {
      if (d_engaged) {
        ::new(address()) T( std::move(other).value() );
      }
   }

   template<class U = T>
      requires (std::is_constructible_v<T, U>
            && !std::is_same_v<std::remove_cvref_t<U>, optional>)
   constexpr
   explicit(!std::is_convertible_v<U, T>)
   optional(U&& arg) {
      do_emplace( std::forward<U>(arg) );
   }

   constexpr ~optional() {
      static_assert(std::is_replaceable_v<optional>
        == std::is_replaceable_v<T>);
      static_assert(
         std::is_trivially_relocatable_v<optional>
        == std::is_trivially_relocatable_v<T>);

      if (d_engaged) {
         d_object.~T();
      }
   }

   constexpr optional& operator=(optional const & rhs);

   constexpr optional& operator=(optional && rhs)
             noexcept(std::is_nothrow_move_assignable_v<T>
                   && std::is_nothrow_move_constructible_v<T>) {
      std::cout << "Assignment\n";
      if (!d_engaged) {
        if (rhs.d_engaged) {
           do_emplace( std::move(rhs.value()) );
        }
      }
      else if (!rhs.d_engaged) {
         d_object.~T();
         d_engaged = false;
      }
      else {
        value() = rhs.value();
      }
      return *this;
   }

   template<class U = T>
```

```
   constexpr optional& operator=(U && arg) {
      std::cout << "Assignment\n";
      if (!d_engaged) {
        do_emplace( std::forward<U>(arg) );
      }
      else {
         d_object = std::forward<U>(arg);
      }
      return *this;
   }

   constexpr T const * operator->() const  noexcept
   { assert(d_engaged); return address(); }

   constexpr T       * operator->()        noexcept
   { assert(d_engaged); return address(); }

   constexpr T const & operator*() const & noexcept
   { assert(d_engaged); return d_object; }
   constexpr T       & operator*()       & noexcept
   { assert(d_engaged); return d_object; }
   constexpr T      && operator*()      && noexcept
   { assert(d_engaged); return std::move(d_object); }
   constexpr T const&& operator*() const&& noexcept
   { assert(d_engaged); return std::move(d_object); }

   constexpr explicit operator bool() const noexcept
   { return d_engaged; }
   constexpr bool         has_value() const noexcept
   { return d_engaged; }

   constexpr T const & value() const &
   { assert(d_engaged); return d_object; }
   constexpr T       & value()       &
   { assert(d_engaged); return d_object; }
   constexpr T      && value()      &&
   { assert(d_engaged); return std::move(d_object); }
   constexpr T const&& value() const&&
   { assert(d_engaged); return std::move(d_object); }
};

consteval int number(int n) {
   optional<int> x{n};
   return x.value();
}

int a[number(5uz)];

int main() {
   optional<int> x;
   assert(!x);

   std::cout << "Assignments to x\n";
   x = 3;
   auto y = x;

   x = 4;
   std::cout << "swap x\n";
   std::swap(x, y);

   assert(3 == *x);
   assert(4 == *y);

   optional<std::shared_ptr<int>> p1;

   std::cout << "Assignments to p\n";
   p1 = std::make_shared<int>(3);
   auto p2 = p1;
```

```
   p2 = std::make_shared<int>(4);
   std::cout << "swap p\n";
   std::swap(p1, p2);
}
```

16.4 C++26 implementation using internal array

Using an internal array negates the ability to support `constexpr`, but this implementation strategy is used frequently for similar types in other libraries. Managing both ***trivially relocatable*** and ***replaceable*** properties with an empty member must be done with care since mistakenly disabling both properties is easy to do when intending to disable only one or the other.[4]

```
#include <cassert>
#include <cstddef>
#include <iostream>
#include <memory>
#include <new>
#include <type_traits>
#include <utility>

template <bool triviallyRelocatable,
          bool replaceable>
struct ConditionalProperties {};

template <>
struct ConditionalProperties<false,true> replaceable_if_eligible {
    ~ConditionalProperties(){}
};
template <>
struct ConditionalProperties<true,false> trivially_relocatable_if_eligible {
    ~ConditionalProperties(){}
};
template <>
struct ConditionalProperties<false,false> {
    ~ConditionalProperties(){}
};

static_assert( std::is_trivially_relocatable_v<ConditionalProperties<true,true>>);
static_assert( std::is_trivially_relocatable_v<ConditionalProperties<true,false>>);
static_assert(!std::is_trivially_relocatable_v<ConditionalProperties<false,true>>);
static_assert(!std::is_trivially_relocatable_v<ConditionalProperties<false,false>>);

static_assert( std::is_replaceable_v<ConditionalProperties<true,true>>);
static_assert(!std::is_replaceable_v<ConditionalProperties<true,false>>);
static_assert( std::is_replaceable_v<ConditionalProperties<false,true>>);
static_assert(!std::is_replaceable_v<ConditionalProperties<false,false>>);

template <class T>
class optional trivially_relocatable_if_eligible replaceable_if_eligible {
   alignas (T)
   std::byte d_object[sizeof (T)];
   union {
      bool      d_engaged{false};
      ConditionalProperties<std::is_trivially_relocatable_v<T>,
                            std::is_replaceable_v<T>> enforce_properties;
   };

   constexpr T const * address() const noexcept
   { return reinterpret_cast<T const *>(d_object); };
   constexpr T       * address()       noexcept
   { return reinterpret_cast<T       *>(d_object); };

public:
   using value_type = T;
```

```
// 22.5.3.2, constructors
constexpr optional() noexcept = default;

constexpr optional(optional const & other) : d_engaged{other.d_engaged} {
   if (d_engaged) {
     ::new(address()) T( other.value() );
   }
}

constexpr optional(optional&& other) noexcept(std::is_nothrow_move_constructible_v<T>)
: d_engaged{other.d_engaged}
{
   if (d_engaged) {
     ::new(address()) T( std::move(other).value() );
   }
}

template<class U = T>
   requires (std::is_constructible_v<T, U>
         && !std::is_same_v<std::remove_cvref_t<U>, optional>)
constexpr
explicit(!std::is_convertible_v<U, T>)
optional(U&& arg) {
   ::new(address()) T( std::forward<U>(arg) );
   d_engaged = true;
}

// 22.5.3.3, destructor
constexpr ~optional() {
   static_assert(std::is_trivially_relocatable_v<optional> ==
                 std::is_trivially_relocatable_v<T>);
   static_assert(std::is_replaceable_v<optional> ==
                 std::is_replaceable_v<T>);

   if (d_engaged) {
      address()->~T();
   }
}

// 22.5.3.4, assignment
constexpr optional& operator=(optional const & rhs);

constexpr optional& operator=(optional && rhs)
  noexcept(std::is_nothrow_move_assignable_v<T>
        && std::is_nothrow_move_constructible_v<T>)
{
   std::cout << "Assignment\n";
   if (!d_engaged) {
     if (rhs.d_engaged) {
        ::new(address()) T( std::move(rhs.value()) );
        rhs.d_engaged = false;
        d_engaged = true;
     }
   }
   else if (!rhs.d_engaged) {
      address()->~T();
      d_engaged = false;
   }
   else {
     value() = rhs.value();
   }
   return *this;
}

template<class U = T>
constexpr optional& operator=(U && arg) {
   std::cout << "Assignment\n";
   if (!d_engaged) {
```

```
        ::new(address()) T( std::forward<U>(arg) );
        d_engaged = true;
      }
      else {
         *address() = std::forward<U>(arg);
      }
      return *this;
   }


   // 22.5.3.7, observers
   constexpr T const * operator->() const  noexcept
   { assert(d_engaged); return  address(); }
   constexpr T       * operator->()        noexcept
   { assert(d_engaged); return  address(); }

   constexpr T const & operator*() const & noexcept
   { assert(d_engaged); return *address(); }
   constexpr T       & operator*()       & noexcept
   { assert(d_engaged); return *address(); }
   constexpr T      && operator*()      && noexcept
   { assert(d_engaged); return std::move(*address()); }
   constexpr T const&& operator*() const&& noexcept
   { assert(d_engaged); return std::move(*address()); }

   constexpr explicit operator bool() const noexcept
   { return d_engaged; }
   constexpr bool         has_value() const noexcept
   { return d_engaged; }

   constexpr T const & value() const &
   { assert(d_engaged); return *address(); }
   constexpr T       & value()       &
   { assert(d_engaged); return *address(); }
   constexpr T      && value()      &&
   { assert(d_engaged); return std::move(*address()); }
   constexpr T const&& value() const&&
   { assert(d_engaged); return std::move(*address()); }
};

int main() {
   optional<int> x;
   assert(!x);

   std::cout << "Assignments to x\n";
   x = 3;
   auto y = x;

   x = 4;
   std::cout << "swap x\n";
   std::swap(x, y);

   assert(3 == *x);
   assert(4 == *y);

   optional<std::shared_ptr<int>> p1;

   std::cout << "Assignments to p\n";
   p1 = std::make_shared<int>(3);
   auto p2 = p1;

   p2 = std::make_shared<int>(4);
   std::cout << "swap p\n";
   std::swap(p1, p2);
}
```

§ 17 Proposed Wording

Make the following changes to the C++ Working Draft. All wording is relative to [N4993], the latest working draft at the time of writing.

§ 17.1 Add new identifiers with a special meaning

§ 5.11 [lex.name] Identifiers

Table 4: Identifiers with special meaning [tab:lex.name.special]

final	import	module	override	replaceable_if_eligible	trivially_relocatable_if_eligible

§ 17.2 Specify trivially relocatable types

Editorial note: *We have separated each sentence to improve clarity rather than trying to identify the definition of so many terms as a single paragraph.*

§ 6.8.1 [basic.types.general] General

9 Arithmetic types (6.8.2 [basic.fundamental]), enumeration types, pointer types, pointer-to-member types (6.8.4 [basic.compound]), `std::nullptr_t`, and cv-qualified (6.8.5 [basic.type.qualifier]) versions of these types are collectively called *scalar types*.

Scalar types, trivially copyable class types (11.2 [class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *trivially copyable types*.

Scalar types, trivial class types (11.2 [class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *trivial types*.

Scalar types, trivially relocatable class types (11.2 [class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *trivially relocatable types*.

Scalar types, replaceable class types (11.2 [class.prop]), and arrays of such types are collectively called *replaceable types*.

Scalar types, standard-layout class types (11.2 [class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called *standard-layout types*.

Scalar types, implicit-lifetime class types (11.2 [class.prop]), array types, and cv-qualified versions of these types are collectively called *implicit-lifetime types*.

§ 17.3 Address trivial relocation of lambdas

§ 7.5.6.2 [expr.prim.lambda.closure] Closure types

2 The closure type is declared in the smallest block scope, class scope, or namespace scope that contains the corresponding *lambda-expression*.

[*Note 1:* This determines the set of namespaces and classes associated with the closure type (6.5.4 [basic.lookup.argdep]). The parameter types of a *lambda-declarator* do not affect these associated namespaces and classes. —*end note*]

3 The closure type is not an aggregate type (9.4.2 [dcl.init.aggr]); it is a structural type (13.2 [temp.param]) if and only if the lambda has no *lambda-capture*. An implementation may define the closure type differently from what is described below provided this does not alter the observable behavior of the program other than by changing:

(3.1) — the size and/or alignment of the closure type,

(3.2) — whether the closure type is trivially copyable (11.2 [class.prop]), or

(3.x) — whether the closure type is trivially relocatable (11.2 [class.prop]), or

(3.y) — whether the closure type is replaceable (11.2 [class.prop]), or

(3.3) — whether the closure type is a standard-layout class (11.2 [class.prop]).

An implementation shall not add members of rvalue reference type to the closure type.

§ 17.4 Update grammar to support `_if_eligible` contextual keywords

§ 11.1 [class.pre] Preamble

1 A class is a type. Its name becomes a class-name (11.3 [class.name]) within its scope.

class-name :
identifier
simple-template-id

A *class-specifier* or an *elaborated-type-specifier* (9.2.9.5 [dcl.type.elab]) is used to make a *class-name*. An object of a class consists of a (possibly empty) sequence of members and base class objects.

class-specifier :
class-head `{` *member-specification*$_{opt}$ `}`

class-head:
class-key attribute-specifier-seq$_{opt}$ *class-head-name class-*~~*virt*~~*property-specifier*$_{opt}$ *base-clause*$_{opt}$
class-key attribute-specifier-seq$_{opt}$ *base-clause*$_{opt}$

class-head-name :
nested-name-specifier$_{opt}$ *class-name*

class-property-specifier-seq:
class-property-specifier class-property-specifier-seq$_{opt}$

class-~~*virt*~~*property-specifier* :
`final`
`replaceable_if_eligible`
`trivially_relocatable_if_eligible`

class-key :
`class`
`struct`
`union`

A class declaration where the *class-name* in the *class-head-name* is a *simple-template-id* shall be …

4 [*Note 2:* The *class-key* determines whether the class is a union (11.5 [class.union]) and whether access is public or private by default (11.8 [class.access]). A union holds the value of at most one data member at a time. —*end note*]

5 If a class is marked with the *class-virt-specifier* `final` and it appears as a *class-or-decltype* in a base-clause (11.7 [class.derived]), the program is ill-formed. Whenever a *class-key* is followed by a *class-head-name*, the identifier `final`, and a colon or left brace, `final` is interpreted as a *class-virt-specifier*.

5 The same *class-property-specifier* shall not appear multiple times within a single *class-property-specifier-seq*.

Whenever a *class-key* is followed by a *class-head-name*, one of the identifiers `final`, `replaceable_if_eligible`, or `trivially_relocatable_if_eligible`, and a colon or left brace, the identifier is interpreted as a *class-property-*

specifier.

[*Example 2:*

```
struct A;
struct A final {}; // OK, definition of struct A,
                   // not value-initialization of variable final

struct X {
  struct C { constexpr operator int() { return 5; } };
  struct B ~~final~~trivially_relocatable_if_eligible : C{};
                                    // OK, definition of nested class B,
                                    // not declaration of a bit-field
                                    // member ~~final~~trivially_relocatable_if_eligible
};
```

—*end example*]

u If a class is marked with the *class-property-specifier* `final` and that class appears as a *class-or-decltype* in a base-clause (11.7 [class.derived]), the program is ill-formed.

6 [*Note 3:* Complete objects of class type have nonzero size. Base class subobjects and members declared with the `no_unique_address` attribute (9.12.12 [dcl.attr.nouniqueaddr]) are not so constrained. —*end note*]

§ 17.5 Specification for trivially relocatable classes

Design note:
Declaring a class as trivially relocatable is possible, by means of the `trivially_relocatable_if_eligible` *specifier, even if that class has user-provided special members. Note that such a declaration is not permitted to break the encapsulation of members or bases and allow for their trivial relocation when they, themselves, are not trivially relocatable.*

§ 11.2 [class.prop] Properties of classes

2 A *trivial class* is a class that is trivially copyable and has one or more eligible default constructors (11.4.5.2 [class.default.ctor]), all of which are trivial.

[*Note 1:* In particular, a trivially copyable or trivial class does not have virtual functions or virtual base classes. —*end note*]

a A class is *eligible for trivial relocation* unless it has

- — any virtual base classes, or
- — a base class that is not a trivially relocatable class, or
- — a non-static data member of a non-reference type that is not of a trivially relocatable type, or
- — a deleted destructor.

b A class C is *eligible for replacement* unless it has

- — a base class that is not a replaceable class, or
- — a non-static data member that is not of a replaceable type,
- — no eligible constructor that would be selected when an object of type `C` is direct-initialized from an xvalue of type `C`,
- — no eligible assignment operator that would be selected when an object of type `C` is assigned from an xvalue of type `C`,
- — no destructor.

c A class `C` is a *trivially relocatable class* if it is eligible for trivial relocation and

— has the `trivially_relocatable_if_eligible` *class-property-specifier*, or

— is a union with no user-declared special member functions, or

— satisfies all of the following:

— when an object of type `C` is direct-initialized from an xvalue of type `C`, overload resolution would select a constructor that is neither user-provided nor deleted, and

— when an xvalue of type `C` is assigned to an object of type `C`, overload resolution would select an assignment operator that is neither user-provided nor deleted, and

— it has a destructor that is neither user-provided nor deleted.

d [*Note 2:* Accessibility of the special member functions is not considered when establishing trivial relocatability. — *end note*]

e [*Note 3:* A type with non-static data members that are const-qualified or are references can be trivially relocatable. —*end note*]

f [*Note 4:* Trivially copyable classes are trivially relocatable unless they have deleted special members. —*end note*]

g A class `C` is a *replaceable class* if it is eligible for replacement and

— has the `replaceable_if_eligible` *class-property-specifier*, or

— is a union with no user-declared special member functions, or

— satisfies all of the following:

— when an object of type `C` is direct-initialized from an xvalue of type `C`, overload resolution would select a constructor that is neither user-provided nor deleted, and

— when an xvalue of type `C` is assigned to an object of type `C`, overload resolution would select an assignment operator that is neither user-provided nor deleted, and

— it has a destructor that is neither user-provided nor deleted.

h [*Note 5:* Accessibility of the special member functions is not relevant. —*end note*]

i [*Note 6:* Trivially copyable classes are replaceable unless they have deleted special members. —*end note*]

3 A class `S` is a *standard-layout class* if it:

(3.1) …

§ 17.6 Add feature macros

Add a `__cpp_trivial_relocatability` feature-test macro to the table in 15.11 [cpp.predefined], set to the date of adoption.

…

§ 17.7 Library wording

Design note: *The first paragraph explicitly captures the status quo that these class properties — the whole set specified in 11.2 [class.prop] — are deliberately left as a quality of implementation feature.*

The second paragraph addresses permission to add the new annotation wherever an implementation might find it useful, without being constrained by its absence from the library specification, much like we grant permission to add `noexcept` *specifications to functions of the implementation's choosing. The specification really needs only the second paragraph, but adding a section with the first paragraph gives us somewhere to hang the wording.*

§ 16.4.6.X Properties of library classes [library.class.props]

1 Unless specifically stated, it is unspecified whether any class described in Clause 17 through Clause 34 and Annex D is a trivial class, a trivially copyable class, a trivially relocatable class, a standard-layout class, or an implicit-lifetime class (11.2 [class.prop]).

> **LWG Wording Note**: The concepts of relocation and replacement are described extensively in this paper but are not defined in the wording. Describing the constraints on an implementation's use of `trivially_relocatable_if_eligible` and `replaceable_if_eligible` is, therefore, difficult. LWG might want to reword the blanket permissions below.

2 An implementation may add the *class-property-specifier* `trivially_relocatable_if_eligible` to any class for which trivial relocation would be semantically equivalent to move-construction of the destination object followed by destruction of the source object.

3 An implementation may add the *class-property-specifier* `replaceable_if_eligible` to any class for which move assignment is semantically equivalent to destroying the assigned-to object, then move-constructing a new object in its place.

§ 17.8 Add new type traits

§ 21.3.3 [meta.type.synop] Header <type_traits> synopsis

```
template< class T >
struct is_replaceable;

template< class T >
struct is_trivially_relocatable;

template< class T >
struct is_nothrow_relocatable;

template< class T >
inline constexpr bool is_replaceable_v = is_replaceable<T>::value;

template< class T >
inline constexpr bool is_trivially_relocatable_v = is_trivially_relocatable<T>::value;

template< class T >
inline constexpr bool is_nothrow_relocatable_v = is_nothrow_relocatable<T>::value;
```

§ 21.3.5.4 [meta.unary.prop] Type properties

Template	Condition	Preconditions
`template<class T> struct is_replaceable;`	`T` is a replaceable type (6.8.1 [basic.types.general])	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`
`template<class T> struct is_trivially_relocatable;`	`T` is a trivially relocatable type (6.8.1 [basic.types.general])	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`
`template<class T> struct is_nothrow_relocatable;`	`is_trivially_relocatable_v<T> || is_nothrow_move_constructible_v<T>`	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`

§ 17.9 Specify the compiler-magic functions

Add to the <memory> header synopsis in 20.2.2 [memory.syn]p3.

§ 20.2.2 [memory.syn] Header <memory> synopsis

```
// 20.2.6, explicit lifetime management
template<class T>
```

```
  T* start_lifetime_as(void* p) noexcept;                                    // freestanding
template<class T>
  const T* start_lifetime_as(const void* p) noexcept;                        // freestanding
template<class T>
  volatile T* start_lifetime_as(volatile void* p) noexcept;                  // freestanding
template<class T>
  const volatile T* start_lifetime_as(const volatile void* p) noexcept;      // freestanding
template<class T>
  T* start_lifetime_as_array(void* p, size_t n) noexcept;                    // freestanding
template<class T>
  const T* start_lifetime_as_array(const void* p, size_t n) noexcept;        // freestanding
template<class T>
  volatile T* start_lifetime_as_array(volatile void* p, size_t n) noexcept;  // freestanding
template<class T>
  const volatile T* start_lifetime_as_array(const volatile void* p,
                                            size_t n) noexcept;              // freestanding
```

```
template <class T>
  T* trivially_relocate(T* first, T* last, T* result);                       // freestanding

template <class T>
  constexpr T* relocate(T* first, T* last, T* result);                       // freestanding
```

§ 20.2.6 [obj.lifetime] Explicit lifetime management

```
template <class T>
  T* trivially_relocate(T* first, T* last, T* result);
```

a *Mandates:* `T` is a complete type, and `is_trivially_relocatable_v<T> && !is_const_v<T>` is `true`.

b *Preconditions:*

(b.1) — [`first`, `last`) is a valid range.

(b.2) — [`result`, `result + (last - first)`) denotes a region of storage that is a subset of the region of storage reachable through (6.8.4 [basic.compound]) `result` and suitably aligned for the type `T`.

(b.3) — `(last - first) != 1`, or `*first` points to a complete object (6.7.2 [intro.object]).

c *Postconditions:*

No effect if `result == first`.

Otherwise, the range denoted by [`result`, `result + (last - first)`) contains objects (including subobjects) whose lifetime has begun and whose object representations are the original object representations of the corresponding objects in the source range [`first`, `last`). If any of the aforementioned objects is a union, its active member is the same as that of the corresponding union in the source range. If any of the aforementioned objects has a non-static data member of reference type, that reference refers to the same entity as does the corresponding reference in the source range. The lifetime of the original objects in the source range has ended.

d *Returns:* `result + (last - first)`.

e *Throws:* Nothing.

f *Complexity:* Linear in the length of the source range.

g *Remarks:* No constructors or destructors are invoked.

```
template <class T>
  constexpr T* relocate(T* first, T* last, T* result);
```

u *Mandates*: `is_nothrow_relocatable_v<T> && !is_const_v<T>` is `true`.

v *Preconditions*: `(last - first) != 1`, or `*first` points to a complete object (6.7.2 [intro.object]).

w *Effects*: No effect if `result == first`; otherwise, if not called during constant evaluation and `T` is trivially relocatable, then has effects equivalent to `trivially_relocate(first, last, result)`; otherwise, for each element in [`first`, `last`), move constructs that element to the corresponding location in [`result`, `result + (last - first)`) and then invokes that element's destructor.

x *Remarks:* Overlapping ranges are supported.

y *Returns:* `result + (last-first)`.

z *Throws*: Nothing.

17.9.1 Feature-test macro

Add a new `__cpp_lib_trivially_relocatable` feature-test macro in [version.syn]:

```
#define __cpp_lib_trivially_relocatable 20XXXXL // also in <memory>, <type_traits>
```

18 Acknowledgements

This document is written in Markdown and depends on the extensions in `pandoc` and `mpark/wg21`, and we would like to thank the authors of those extensions and associated libraries.

The authors would also like to thank Brian Bi for his assistance in proofreading this paper, especially the proposed Core wording. Additional thanks to Jens Maurer who helped to greatly refine the wording in advance of its first Core review.

Additional thanks are due to Giuseppe D'Angelo for clearly articulating weaknesses in our earlier proposals and working with us to improve the specification and to Louis Dionne for authoring [P3516R0], which allows for a complete consideration of the consumer interface for relocation.

Also, this paper has been greatly improved by feedback from Arthur O'Dwyer, author of [P1144], who corrected many bad assumptions we made about his paper and helped bring the technical differences into focus. We also benefited from several examples he shared to help illustrate those differences and misunderstandings.

19 References

[CWG1734] James Widman. 2013-08-09. Nontrivial deleted copy functions.
https://wg21.link/cwg1734

[N4993] Thomas Köppe. 2024-10-16. Working Draft, Programming Languages — C++.
https://wg21.link/n4993

[P1144] Arthur O'Dwyer. std::is_trivially_relocatable.
https://wg21.link/p1144

[P2786R10] Pablo Halpern, Alisdair Meredith, Joshua Berne, Corentin Jabot, Pablo Halpern, Lori Hughes. 2024-11-21. Trivial Relocatability For C++26.
https://wg21.link/p2786r10

[P2786R11] Pablo Halpern, Joshua Berne, Corentin Jabot, Pablo Halpern, Lori Hughes. 2024-12-17. Trivial Relocatability For C++26.
https://wg21.link/p2786r11

[P2786R6] Mungo Gill, Alisdair Meredith. 2024-05-21. Trivial Relocatability For C++26.
https://wg21.link/p2786r6

[P2786R9] Pablo Halpern, Alisdair Meredith, Joshua Berne, Corentin Jabot, Pablo Halpern, Lori Hughes. 2024-11-16. Trivial Relocatability For C++26.
https://wg21.link/p2786r9

[P2839R0] Brian Bi, Joshua Berne. 2023-05-15. Nontrivial relocation via a new "owning reference" type.
https://wg21.link/p2839r0

[P2959R0] Alisdair Meredith. 2023-10-15. Container Relocation.
https://wg21.link/p2959r0
[P3239R0] Alisdair Meredith. 2024-05-22. A Relocating Swap.
https://wg21.link/p3239r0
[P3516R0] Louis Dionne and Giuseppe D'Angelo. Uninitialized algorithms for relocation.
https://isocpp.org/files/papers/P3516R0.html

---

1. All citations to the Standard are to working draft N4993 unless otherwise specified.↩

2. Much rationale related to **trivial relocation** can be found in [P2786R6].↩

3. This implementation can be seen compiling on Compiler Explorer here: compiler-explorer.↩

4. This implementation can be seen compiling on Compiler Explorer here: compiler-explorer.↩